

Responsive Test-Time Model Adaptation for Mobile Applications via Runtime-efficient Sparse Updates

Cheng Fang, Bin Guo*, *Senior Member, IEEE*, Sicong Liu*, *Member, IEEE*, Zimu Zhou, *Member, IEEE*, Jiaqi Tang, *Student Member, IEEE*, Ke Ma, Shiyao Luo, Geyang Song, Zhiwen Yu, *Senior Member, IEEE*

Abstract—Adapting the on-device deep learning (DL) models to continual and unpredictable domain shifts is crucial for mobile applications like autonomous driving and augmented reality, which require seamless user experiences in ever-changing environments. Test-time adaptation (TTA) offers a promising solution that fine-tunes models with *unlabeled* real-time data just before making predictions. However, TTA introduces a significant challenge, its forward-backward-reforward pipeline increases latency, compromising responsiveness in time-sensitive mobile scenarios. In this paper, we present AdaShadow, a responsive test-time adaptation framework designed for non-stationary mobile data distributions and resource dynamics. AdaShadow focuses on selectively updating only adaptation-critical layers, minimizing the latency impact. While this idea has been explored in general on-device training, the unsupervised and real-time nature of TTA presents unique challenges in assessing layer importance, predicting latency, and planning layer updates efficiently. AdaShadow addresses these challenges through a *backpropagation-free importance assessor* for rapid identification of critical layers, a *unit-based runtime latency predictor* that accounts for resource constraints, and an *online layer update scheduler* to ensure prompt retraining. Furthermore, a memory I/O-aware computation reuse scheme optimizes the reforward pass to further reduce delays. Our evaluations show that AdaShadow reduces adaptation latency by up to $3.7\times$ (achieving ms-level response time) and improves accuracy by up to 25.4%, all while maintaining low memory and energy costs across CNN, and ViT models, outperforming state-of-the-art methods.

Index Terms—Latency-efficient test-time adaptation, mobile environments, mobile computing, ubiquitous computing.

I. INTRODUCTION

Deep learning (DL) models, pre-trained in the cloud, are increasingly being deployed on mobile devices to enable autonomous edge intelligence [1], [2], particularly in regions where cloud streaming is restricted [3]. Examples of such applications include on-device motion tracking (e.g., Google ARcore [4]) and AR/VR platforms (e.g., Apple ARKit [5]). Recently, mobile vision AI applications powered by CNNs and Vision Transformers (ViTs), such as Google Photos [6] and Apple Intelligence [7] have also become widespread.

These DL models must function reliably in dynamic mobile environments, which requires *rapid model adaptation to unexpected domain shifts*. For example, an autonomous vehicle may face temporary road signs or altered traffic signals due to construction and must update its DL models using

a small number of collected samples gathered in real-time [8]. This adaptation is essential for ensuring both safety and operational effectiveness. Likewise, AR headsets must seamlessly integrate virtual elements into the physical world. As factors like lighting, field of view, and background changes with user movement, these devices need to *promptly* adjust their object recognition and tracking models using limited video input, ensuring virtual overlays remain consistent with the changing physical surroundings. For example, a vehicle traveling at 90km/h requires its 60fps traffic sign recognition model to adapt within 16.6ms to maintain safety.

Test-time adaptation (TTA) provides an effective approach to address *unpredictable domain shifts* in mobile environments [9]. It represents an emerging domain adaptation paradigm that (i) leverages unlabeled test data and (ii) operates independently of the source dataset and supervision from pre-training [10]. TTA generally works in three stages. First, a *forward* pass generates initial inference results on the shifted test samples, i.e., the live data sensed in a new environment. If the inference confidence is low (e.g., high entropy), a *backward* pass follows to adjust the DL model parameters using unsupervised loss based on the initial forward results. Finally, a *reforward* pass applies the updated DL model to the same batch of input data to produce the final predictions. This continual, *unsupervised, source-free* adaptation is particularly beneficial for mobile applications, which often do not have access to pre-training data due to privacy or network limitations. Additionally, these applications typically eliminate the possibility of pre-labeling target domains because of the necessary labor and resources.

Despite advances in TTA algorithms [10]–[12], their practical application in mobile settings remains challenging. The *forward-backward-reforward* pipeline, crucial for adapting to domain shifts, introduces significant *latency* compared to standard inference (see § II-B), which undermines the *responsiveness* of *time-sensitive* mobile applications such as AR and autonomous vehicles, where even slight delays can severely affect user experience and operational safety. Although pioneering efforts toward efficient TTA for CNN [12]–[14] and ViT [15] focus on reducing computational or memory costs, these improvements do not easily translate to reduced *wall-clock time*. Backward-free TTA on CNN [16], [17] and ViT [18] suffer from significant accuracy degradation during continual adaptation. This gap highlights the need for *latency-efficient* TTA that enables *fast* and *accurate* adaptation in non-stationary mobile environments.

Corresponding Author: Bin Guo (email: guob@nwpu.edu.cn), Sicong Liu (e-mail: scliu@nwpu.edu.cn)

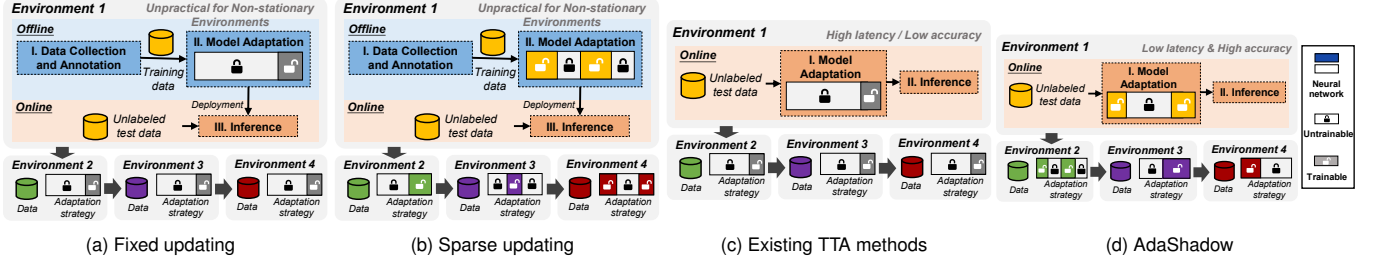


Fig. 1. Comparison of existing work (a, b, c) and AdaShadow.

In this paper, we propose **AdaShadow**, a novel framework for *responsive* test-time adaptation, like how a shadow *swiftly* reacts to the body’s movements. The key principle is *sparse updating*, i.e., selectively refining critical layers to reduce both computational and memory overhead during backpropagation, thereby minimizing latency [19]. While this approach has been explored to enhance the latency efficiency of general DL model training on mobile devices [19], sparse updating in TTA faces distinct challenges due to its *unsupervised* and *online* characteristics. We discuss these challenges in detail below.

- **Challenge #1: identifying critical layers at low latency.** The importance of each layer varies significantly across different data distributions [20]. In non-stationary mobile environments, each data batch may present distinct domain shifts, requiring rapid assessment of layer importance. Previous studies [19], [21] evaluate CNN layer importance by calculating gradients using *labeled* data, which is computationally expensive (i.e., higher latency) and prone to errors in TTA, as test data in highly dynamic mobile scenarios are typically unlabeled. For ViT, [22] measures weight importance by calculating the loss gradient with respect to weight updates, showing each weight’s effect on the loss. For example, ElasticTrainer [19] incurs a 2.98s delay and 14.8% gradient errors when aggregating gradients to determine importance for ResNet50 on unlabeled data.

- **Challenge #2: predicting runtime latency for layer update.** To ensure that the updated model architecture meets to low-latency requirements, it is crucial to accurately predict the runtime latency of each retained layer in the new model. Previous works [19] measure CNN/ViT layer latency offline and then use this as the runtime model inference latency. However, mobile devices are interactive, and their hardware resource availability fluctuates dynamically due to user operations, app switching between foreground and background, and competition for computational and memory resources. This variability makes offline estimation inaccurate for predicting online model inference latency.

- **Challenge #3: online scheduling layer update strategy efficiently.** The architectures of visual DL models are numerous, making it impractical to analyze update latency and develop sparse update strategies for each, hindering efficient management of structural diversity. What’s more, even with precise knowledge of each layer’s importance and runtime latency, identifying the optimal sparse updating strategy to enhance performance under strict delay constraints remains challenging due to the vast search space. Efficient strategies are needed to explore and prune this search space effectively.

AdaShadow addresses these challenges through three key

functional designs.

First, we observe that the *forward* pass in TTA offers a valuable, often-overlooked opportunity to assess layer importance without the extra overhead and inaccuracy of backpropagation gradient computation. Based on this insight, we introduce a *backpropagation-free* layer importance assessor that evaluates the divergence in output feature maps across different environments, dynamically determining layer importance without the need for labeled data.

Second, achieving timely adaptation requires precise and responsive latency feedback, as system *dynamics*, particularly related to hardware and memory, can significantly affect layer update times. To address this, we propose an online unit-based latency predictor that integrates dynamic factors into the static prediction model. We *observe* that the execution latency of DL models units depends on two key factors: on-chip kernel computation and off-chip memory access. While the kernel’s execution strategy and memory allocation remain static, dynamic variables like kernel utilization and cache hit rates can be measured at runtime, enabling real-time latency predictions. Also, this unit-based approach is highly versatile, applicable across various DL models.

Third, using the predictions of unit importance and update latency, we develop a lightweight dynamic programming (DP)-based online scheduler. We observe that the update latency of visual DL models can be summarized by a limited set of rules and abstract the models into five types of units based on their update latency patterns and analyze each type. Based on split units, then the scheduler efficiently identifying the optimal layer update strategy across different architectures for the three-stage TTA process, defining subproblems, utilizing recursion, and excluding invalid ones. Notably, **AdaShadow** complements existing CNN/ViT adaptation techniques such as LoRA [23] by applying its efficient adaptation strategies only to the critical layers selected by **AdaShadow**, further enhancing model adaptation efficiency.

In the implementation, **AdaShadow** optimizes the TTA loss for *small batches*, which is typical in mobile applications, and utilizes *computation reuse* between the forward and reforward passes to enhance performance. We evaluate **AdaShadow** across three real-world mobile scenarios, addressing over 30 types of data shifts and 5 types of resource dynamics, using 3 mobile devices with different hardware architectures and 6 models, including CNN and ViT architectures. The results demonstrate that **AdaShadow** reduces adaptation latency by up to 72.9% and improves accuracy by up to 25.4% compared to state-of-the-art generic/efficient TTA and resource-efficient supervised adaptation methods. In scenarios with continual shifts, **AdaShadow** achieves up to a 94.6% improvement in

latency (§ VII). Our main contributions are as follows:

- To the best of our knowledge, this is the first work on near-real-time on-device CNN/ViT adaptation in non-stationary mobile environments without access to labels or source data. It resolves the latency challenges of TTA in mobile contexts without sacrificing accuracy.
- We propose **AdaShadow**, a comprehensive system design featuring a backpropagation-free layer importance assessor, a runtime latency predictor, an online update scheduler, and optimizations for efficient, resource-aware TTA. **AdaShadow** integrates smoothly with mainstream TTA pipelines and supports various DL model architectures.
- The experiments show that **AdaShadow** outperforms existing research on generic TTA [10], [11], efficient TTA [12]–[14], and efficient training (with labels) [19], [21], [24] in balancing accuracy and latency with low memory and energy consumption across diverse tasks, shifts, and devices.

II. OVERVIEW

A. Primer on Test-time Adaptation (TTA)

Test-time adaptation (TTA) is an emerging *unsupervised domain adaptation* paradigm designed to counteract *domain shifts* [9]. It adapts DL models *pre-trained* in the source domain to *unlabeled* target data during *testing*, enhancing accuracy on target data [10]. What sets TTA apart is its assumption of no access to *source data* or *supervision* from the pre-training stage, making it ideal for mobile applications where (i) accessing source data is impractical due to privacy concerns or bandwidth limitations on mobile devices, and (ii) annotating target domain is infeasible or resource-intensive.

However, TTA often introduces significant latency (see details in § II-B) or risks accuracy declines due to overfitting to new environments [11], [13]. In light of these limitations, this paper investigates *responsive* test-time adaptation of DL models to *non-stationary mobile* domain shifts, with a focus on **latency efficiency**. Many mobile applications demand *real-time responsiveness*, requiring DL models to adapt swiftly, delivering accurate inference under continuous domain shifts without noticeable delays. Specifically, the non-stationary mobile environment requires the adaptation process to have the following two key capabilities:

- (i) operate effectively with *small batches* of mobile sensor data; For instance, a car-mounted camera’s image sensor only collects 4 frames (a batch) in a 100-ms adaptation window, which is $64\times$ smaller than the batch size used in cloud-based ViT-B/16 pre-training [25].
- (ii) account for *runtime resource dynamics*. Unlike resource-rich cloud environments, fluctuating mobile resource availability greatly impacts the selection of sparse update strategies, such as layer importance and latency. The adaptation must be resource-efficient, minimizing overhead while maintaining device performance, enabling sustained operation over extended periods.

Both requirements present additional challenges when optimizing TTA latency (see § I).

TABLE I
ACCURACY AND LATENCY OF STATE-OF-THE-ART TTA METHODS
AGAINST INFERENCE W/O ADAPTATION ON RESNET50 AND ViT-B/16.

Model	Method	NICO++			CIFAR10-C		
		ACC.(%)	Latency(ms)	Th.(fps)	ACC.(%)	Latency(ms)	Th.(fps)
ResNet50	Source	60.7	10.2	98.2	52.2	9.8	102.2
	Tent [10]	87.8	33.2	30.1	77.8	25.8	38.7
	EATA [19]	88.4	36.5	27.4	79.2	27.5	36.4
	EcoTTA [14]	89.2	42.0	23.8	80.1	31.1	32.2
ViT-B/16	Source	73.8	30.2	33.1	69.1	25.8	38.8
	Tent [10]	90.5	68.5	14.6	85.3	80.0	12.5
	EATA [19]	93.7	98.0	10.2	85.5	77.5	12.9
	EcoTTA [14]	94.2	102.0	9.8	86.8	97.1	10.3

Fig. 1 shows the difference between our work and other representative studies. First, **AdaShadow** adapts the model online in new scenarios without requiring offline data preparation or training phase, thereby enhancing the working efficiency of mobile devices in non-stationary environments. Second, **AdaShadow** adjusts the update strategy across the entire model search space more flexibly compared to existing TTA methods, achieving an optimal balance between adaptation latency and accuracy.

B. Latency Analysis of TTA

Inefficiency of Prior Arts. Despite the growing research focused on improving data efficiency [13] and memory efficiency [12], [14] in test-time adaptation, the challenge of latency remains a less-explored challenge. As shown in Tab. I, EcoTTA [14], a state-of-the-art method, can be up to $4.1\times$ slower than inference with the pre-trained model without adaptation (denoted as *source* in the table), even though it achieves higher accuracy on drifted test datasets. The unsatisfactory latency motivates us to zoom into the latency of TTA.

Latency Bottleneck. Latency is related to computing, memory access, and the availability of hardware resources [26]. A typical TTA pipeline [9] includes three phases: *forward*, *backward*, and *reforward*. In the first two phases (forward and backward), the model updates its parameters via standard gradient descent, based on the input batch data. In the last phase (reforward), the model performs inference on the *same* batch of data utilizing freshly updated parameters. This pipeline, known as the *sequential adaptation/inference* mode¹, prioritizes inference accuracy by adapting the model before making predictions on it. However, it also implies that the inference for any batch of data must wait for the completion of adaptation, leading to noticeable latency. Formally, the overall TTA latency can be calculated as:

$$T = T_a + T_{re} = T_f + T_b + T_{re} \quad (1)$$

where T_a and T_{re} are the latency of model adaptation and inference (reforward), and T_a is further decomposed into T_f and T_b , i.e., latency of forward and backward, respectively.

Fig. 2 shows the latency breakdown of EcoTTA [14] across four diverse model architectures on the NVIDIA Jetson NX, an off-the-shelf edge device. Notably, the *backward latency* (T_b) constitutes over 50% of the overall delay, making it the

¹A few studies [10], [13] prioritize latency over accuracy in process scheduling by only applying the newly adapted model to subsequent sample inference, thus eliminating the reforward latency. Our method achieves higher accuracy with lower latency than this execution mode (see § VII-F).

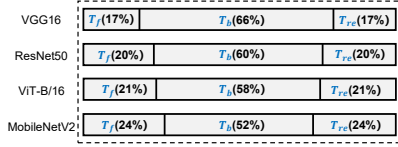


Fig. 2. Latency breakdown of EcoTTA [14], a classic test-time adaptation scheme.

primary *bottleneck* in both CNN and ViT architectures. This bottleneck arises because, while the forward pass computes activations, the backward pass must compute gradients for both *activations* and *parameters*, effectively doubling the computation load and requiring extra memory access to store and retrieve forward pass activations [27]. We observe that the sources of backward propagation latency bottlenecks in CNNs and ViTs are as follows:

- In **CNN**, the primary source of backward latency arises from the convolutional layers. During backpropagation, gradient computation requires convolving both the input feature maps and filters, leading to high computational overhead. Each filter's gradient calculation necessitates separate convolutions for all input feature maps, which increases processing time. As CNNs deepen, the number of parameters in later layers increases, requiring frequent memory accesses to retrieve activations, exacerbating the bottleneck in backward propagation due to computational complexity and high memory bandwidth usage.
- In **ViT**, the main bottleneck in backward propagation is the self-attention mechanism, which has quadratic complexity with respect to sequence length. The self-attention calculation is expressed as: $A_{out}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ where Q , K , and V are the query, key, and value matrices, and d_k is the dimensionality of the query and key vectors. Backpropagation through these matrix multiplications and the softmax operation incurs significant computation. Multi-head attention further amplifies this complexity, as each head computes gradients independently, increasing memory and computation overhead. In vision tasks with standard 14×14 or 16×16 input patches, the backward pass is also impacted by fully connected layers in feed-forward network, where the computational load scales quadratically with the model's dimension, contributing further to overall latency.

C. Problem Statement

Our primary strategy to reduce adaptation latency is *sparse updating* [19], [21], *i.e.*, selectively updating a subset of layers that are crucial for TTA. It reduces the *computation cost of gradients* and the *memory accesses to retrieve intermediate activations*, thereby lowering latency in the backward pass. However, we observe that *updating fewer layers does not guarantee a proportional decrease in TTA latency on mobile devices*. As illustrated in Fig. 3, the latency for updating the first layer is comparable to updating the last four layers. This is because updating the first layer still necessitates *computing gradients* for all layers, leading to significant computation and memory access overhead.

Formally, we explore low-latency test-time adaptation to the unseen mobile environment e via sparse updating by

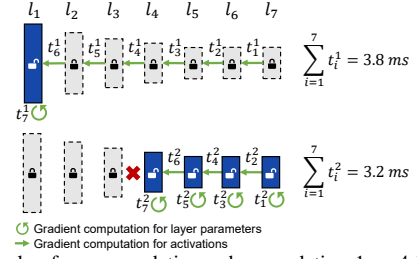


Fig. 3. Example of sparse updating, where updating 1 or 4 layers results in almost the same latency.

formulating the following constrained optimization problem.

$$\max \vec{S}_e \cdot \vec{A}_e \quad s.t. \quad T_f + T'_b(\vec{S}_e) + T'_{re}(\vec{S}_e) \leq \sigma \cdot T \quad (2)$$

where $\vec{S}_e^*(N)$ is the optimal sparse updating strategy, which is a binary vector of length N (the number of layers), and $\vec{A}_e$ is the layer importance vector of the same format as $\vec{S}_e^*(N)$. T'_b and T'_{re} are the optimized backward and reforward latency under strategy $\vec{S}_e^*(N)$, respectively. T is the overall latency without sparse updating, and σ is an expected acceleration factor. Note that T_f is constant with diverse sparse update strategies.

We observe that CNN and ViT share key characteristics that enable the effective application of the sparse updating strategy. Both models consist of layered structures, allowing for the evaluation of individual layer importance and facilitating selective updates. Additionally, they utilize backpropagation for gradient computation, enabling similar optimization strategies during the backward pass. By prioritizing layers that significantly impact performance, we can reduce unnecessary computations in both architectures. This shared structure and the analogous roles of layers in representation learning provide a potential common objective for effectively reducing adaptation latency across CNN and ViT.

Sparse updating primarily reduces the backward latency, yet it also offers opportunities to decrease the reforward latency, as will be explained in § VI. We formulate latency as a constraint with a tunable acceleration factor for easy configuration of the real-time requirements in diverse mobile applications with streaming data, *e.g.*, video processing [28] (image classification, object detection, scene understanding) of vehicle cameras or AR headsets [29] and ViT-based pedestrian detection in autonomous driving scenarios [30].

D. System Overview

Fig. 4 illustrates the system architecture of AdaShadow, which comprises three key functional modules: (i) the *backpropagation-free layer importance assessor*, addressing Challenge #1, evaluates layer importance by measuring the *divergence* in output feature embeddings across different environments, utilizing *unlabeled data* during forward pass (§ III); (ii) the *runtime layer latency predictor*, overcoming Challenge #2, incorporates mobile resource dynamics into latency modeling for online adjustment of offline latency estimates (§ IV); (iii) the *online layer update scheduler*, resolving Challenge #3, employs a lightweight dynamic programming approach to efficiently search for the optimal layer update strategy (§ V). Implementation and additional optimizations of AdaShadow workflow are detailed in § VI.

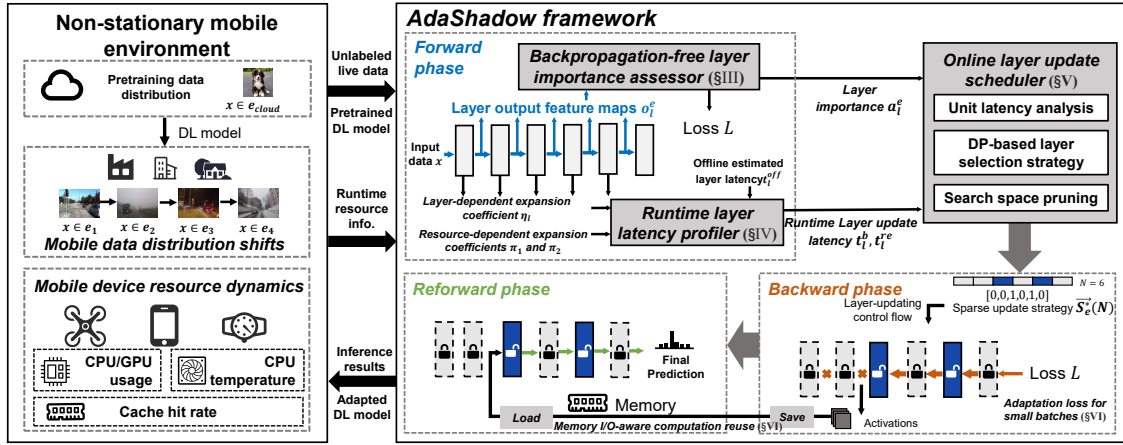


Fig. 4. AdaShadow overview.

III. BACKPROPAGATION-FREE LAYER IMPORTANCE ASSESSOR

A prerequisite for sparse updating is efficient layer importance estimation for on-device model adaptation. This process must be *latency-* and *memory-efficient* to support frequent updates on low-resource mobile devices in non-stationary environments. This subsection introduces a backpropagation-free approach that improves upon prior methods [19], [21] for layer importance assessment, enabling unified evaluation across both CNN and ViT while optimizing adaptation without the overhead of traditional backpropagation.

A. Limitations of Prior Arts

Existing studies [19], [21] rely on *gradients* to assess layer importance on adaptation accuracy, which faces two drawbacks. (i) Accurate *gradients* hinge on the availability of *labels*, which are absent in mobile-end TTA. (ii) More critically, even with access to labels, the dependency on *gradients* to evaluate layer importance incurs a high computational and memory overhead. In **CNN**, reliance on gradients complicates layer importance evaluation due to extensive convolutional operations, resulting in high resource consumption. In **ViT**, this issue is exacerbated by the self-attention mechanism, where backpropagation involves complex matrix multiplications, increasing the computational burden and making traditional gradient-based methods impractical for on-device adaptation.

As shown in Tab. II, TTE [21] and ElasticTrainer [19], two state-of-the-art sparse updating frameworks, incur a latency of up to 167.2s and 2.98s to adapt ResNet50 on Raspberry Pi, and consume over 2,108 MB memory to store intermediate activations for backpropagation. Such overheads are unacceptable for *near-/real-time* DL model adaptation on mobile devices.

B. Principles and Designs

We propose to assess layer importance with *unlabeled data* merely in the *forward pass* for responsiveness by measuring the *divergence* between the layers' *output feature maps* in different environments. The idea is inspired by the distribution alignment techniques [31], which quantifies drifts between the source and target data via the distributions of intermediate

TABLE II
OVERHEAD OF LAYER IMPORTANCE ASSESSORS EVALUATED ON RASPBERRY PI.

Methods	Latency (s)		Memory usage (MB)	
	ResNet50	MobileNetV2	ResNet50	MobileNetV2
TTE [21]	167.24	65.31	2108	695
ElasticTrainer [19]	2.98	1.29	2108	695
AdaShadow	0.11	0.09	0.0086	0.0083

features in the forward stage. The *rationale* is that when a DL model encounters domain shifts from a new environment, it will be less confident about its predictions, as reflected by a deviation in the layer's output distribution from that observed in historical environments.

We cater this principle for TTA context as follows. Given an input data batch x_e from a new environment e , we assess the importance a_l^e of layer l in adapting to x_e by measuring the Kullback-Leibler (KL) divergence between the embedding E_l^e of layer l 's output feature o_l^e on x_e , i.e., $E_l^e = g(o_l^e)$, and the embedding E_l^H on the average output features on historical environments H . Specifically, the layer importance a_l^e is estimated as: $a_l^e = D_{KL}(g(o_l^H) \| g(o_l^e))$ where $D_{KL}(\cdot)$ denotes the KL divergence.

We justify our layer importance metric as follows.

- The *KL divergence* effectively quantifies the data distribution shift in mobile environment e from the history H perceived at layer l . A larger divergence indicates a greater necessity to update layer l . Since we target at adaptation on data streams, e.g., live videos, the domain shifts between adjacent batches tend to be mild. Hence it is unnecessary to adopt computation-intensive metrics, e.g., the Wasserstein distance, which are intended for distributions with low overlaps.
- Rather than the raw output features, we measure the divergence of their *embedding* via an embedding function $g(\cdot)$. It normalizes output features to the same dimensions for fair importance evaluation across layers. More critically, it allows robust estimation with small batches, which is essential for non-stationary environments, as will be discussed next.

C. Optimizations

We implement the layer importance metric with two optimizations tailored for mobile environments:

- For efficient distribution estimation with *small-batch* mobile data, we compute $g(\cdot)$ using only first and second-order moments. Specifically, we calculate channel-wise means $\mu(o_{l,c}^e) = \frac{1}{N} \sum_{c \in \mathcal{C}} o_{l,c}^e$ and variances $\sigma^2(o_{l,c}^e) = \frac{1}{N} \sum_{c \in \mathcal{C}} (o_{l,c}^e - \mu(o_{l,c}^e))^2$ from layer l 's output features $o_{l,c}^e$. The embedding E_l^e is then represented as $[\mu(o_{l,1}^e), \sigma^2(o_{l,1}^e), \dots, \mu(o_{l,N_c}^e), \sigma^2(o_{l,N_c}^e)]$.
- To handle *non-stationary* mobile environments, we update the historical embedding $E_l^H = g(o_l^H)$ via a moving average strategy: $g(o_l^H) = \alpha \cdot g(o_l^e) + (1 - \alpha) \cdot g(o_l^H)$, where $\alpha \in [0, 1]$ is a hyperparameter controlling the rate of incorporating new environment data e .

a_l^e effectively estimates the distribution of output features for both CNN and ViT:

- For **CNN**, which prioritizes *local* spatial features, each channel represents a distinct aspect of the input. The mean $\mu(o_{l,c}^e)$ and variance $\sigma^2(o_{l,c}^e)$ are computed for each channel to capture independent statistical characteristics, enabling precise evaluation of each channel's contribution to the overall feature distribution.
- For **ViT**, the self-attention mechanism emphasizes *global* features, increasing computational complexity for long image patches. By integrating the means and variances of all channels in the embedding E_l^e , we represent the statistical distribution of the entire output layer, capturing both *feature-wise* strength (mean) and dispersion (variance), thus providing a comprehensive statistical framework.

D. Scalability to Diverse Models.

Despite structural differences, both CNN and ViT share a layered architecture, making them suitable for consistent layer importance assessment. Our method efficiently computes channel-wise statistics with small batch sizes typical in mobile environments, optimizing layer importance across both models. By using embedding functions to summarize output features, we mitigate the computational burdens of ViT's self-attention and CNN's convolutional operations. This approach addresses distribution estimation challenges and ensures robust adaptation latency optimization for various models.

As shown in Tab. II, our layer importance assessor reduces evaluation latency by 95% compared to gradient-based schemes, such as TTE [21] and ElasticTrainer [19] for ResNet50 and MobileNetV2. It also uses minimal memory, e.g., less than 8.6 KB for storing the historical embedding. Despite this efficiency, our method maintains high accuracy, as demonstrated in Fig. 5, where it correctly identifies layer importance for TTA on ResNet50 and ViT-B/16 with diverse model sizes. The blue bars indicate the layers selected for updating.

IV. RUNTIME LAYER LATENCY PREDICTOR

Precise layer update latency estimation is another key input to determine the optimal sparse updating strategy. As we target at TTA on mobile devices with limited and dynamic resources, *offline* predicting schemes [19], [26] incur large estimation errors. This subsection introduces an *accurate* and *lightweight*

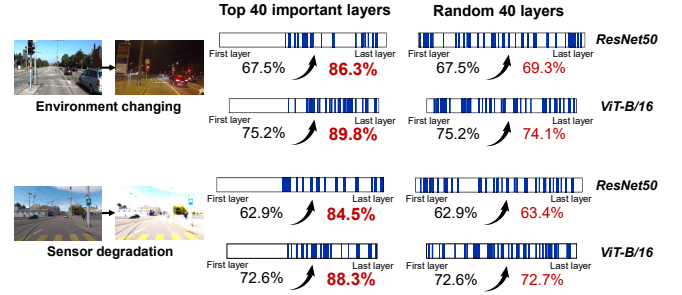


Fig. 5. Accuracy gains when updating top 40 important layers or 40 random layers (blue bars) on ResNet50 and ViT-B/16.

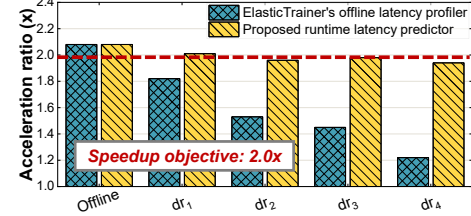


Fig. 6. Comparison of offline/runtime latency predicting with dynamic resource availability.

approach to model and calibrate the backward² latency of individual layers to *runtime* resource dynamics.

A. Observations

We empirically demonstrate the necessity of estimating the training latency at runtime. Specifically, we first apply ElasticTrainer [19], the state-of-the-art sparse training scheme that predicts layer latency offline, to speed up the adaptation of a ResNet50 and NVIDIA Jetson NX by an acceleration ratio of about $2\times$ under *abundant resources* (denoted as *offline* in Fig. 6). Then we simulate four dynamic resource conditions $dr_1 \sim dr_4$ commonly seen in mobile systems [32]: dr_1 : increase the mobile CPU/GPU temperature to 60 °C; dr_2 : add three competing compute-intensive processes; dr_3 : occupy the cache to tune the cache hit rate to be 30%; dr_4 : combine all factors from $dr_1 \sim dr_3$. When running ElasticTrainer [19] in these four mobile system conditions using configurations obtained in the offline case, we observe mild to drastic decrease in acceleration ratios (see Fig. 6). The experiment shows that the *offline layer latency* estimates can be significantly *inaccurate* due to dynamic resource availability. Similarly, running ElasticTrainer in these four dynamic mobile conditions using offline configurations results in up to 60% performance degradation under dr_4 .

Besides the impact of resource availability, we observe that there are significant differences in latency characteristics between CNN and ViT:

- CNN is more affected by changes in computational resources due to competing processes and changes in computation core temperature. This is because convolutional layers usually have low memory latency, but computationally intensive layers, such as fully connected layers, may experience bottlenecks.

²Although we optimize both the latency during backward and reforward (see Equ.(2)), we mainly model and calibrate the estimation on backward latency for two reasons. (i) The backward pass is the latency bottleneck (see § II-B). (ii) In practices, the reforward latency can be easily hidden via pipelining adaptation and inference (see § VII-F).

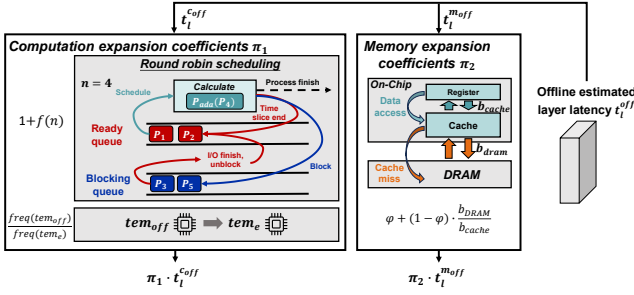


Fig. 7. Layer update latency in offline and online stages.

- ViT is more affected by changes in cache hit rate due to its reliance on memory-intensive operations, particularly in the Q, K, V projections and LayerNorm layers, which are highly sensitive to cache contention and runtime memory bandwidth [33]. Processing long patches requires frequent memory access for key, query, and value retrievals, making the model sensitive to memory bandwidth limitations.

[32] incorporates dynamic resources into latency predictions using a Graph Neural Network (GNN), however, incurring substantial overhead. Furthermore, latency profiling is ineffective without actual execution [34]. In contrast, our proposed method delivers higher acceleration ratios by adapting to these runtime conditions.

Key Observation. We observe that the execution latency of a DL unit whether on CNN, or ViT mainly hinges on on-chip kernel computation and off-chip memory access. Each unit's kernel execution strategy and memory allocation are static, whereas kernel utilization, temperature, and cache-hit-rate are dynamic and measurable at runtime. Moreover, this unit-based approach is versatile across different DL models. For example, basic units in *ResNet* [35] include Conv2d, BatchNorm, and Linear, while in *ViT* [25], they consist of projectors Q, K, V, LayerNorm, and FFN.

B. Our Design

In offline predicting with *abundant, static* resources, the latency t_l^{off} of layer l can be estimated as $t_l^{off} = t_l^{c,off} + t_l^{m,off}$, where $t_l^{c,off}$ and $t_l^{m,off}$ are corresponding latency for computation and memory access profiled offline, respectively. We account for *limited, dynamic* runtime resources on mobile device by modeling the layer latency $t_l = \pi_1 \cdot t_l^{c,off} + \pi_2 \cdot t_l^{m,off}$, where $\pi_1 > 1$ and $\pi_2 > 1$ are the computation and memory expansion coefficients. As measuring $t_l^{c,off}$ and $t_l^{m,off}$ separately is challenging, we correlate t_l to t_l^{off} as follows.

$$t_l = \left(\pi_1 \cdot \frac{\eta_l}{\eta_l + 1} + \pi_2 \cdot \frac{1}{\eta_l + 1} \right) \cdot t_l^{off} \quad (3)$$

where $\eta_l = \frac{t_l^{c,off}}{t_l^{m,off}}$. Note that t_l^{off} is measurable offline. η_l is a layer-dependent hyperparameter but is independent of runtime resource availability, which can also be estimated offline. The two expansion coefficients π_1 and π_2 characterize the penalty of limited resources on computation and memory latency, which are the same across layers and can be profiled at runtime. We explain how to measure the three hyperparameters below.

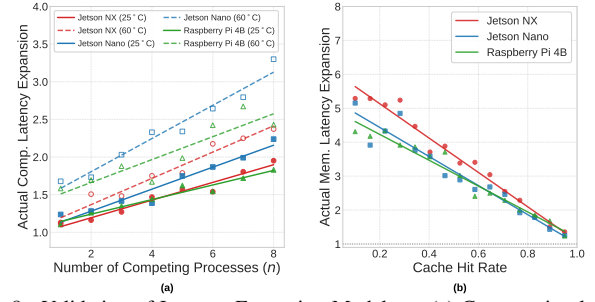


Fig. 8. Validation of Latency Expansion Models on (a) Computation latency and (b) Memory latency.

- **Profiling π_1 .** The computation expansion coefficient π_1 captures the latency overhead caused by resource contention and thermal throttling. Theoretically, this is grounded in the *processor sharing* mechanism of OS schedulers [36] and hardware *Dynamic Voltage and Frequency Scaling (DVFS)* [37]. Specifically, π_1 depicts two runtime factors:

- The OS kernel of the mobile CPU and GPU often employs round robin scheduling (see Fig. 7) [38] under the processor sharing mechanism. An increase in process number leads to a linear growth in process waiting time.
- Dynamic Voltage Frequency Scaling (DVFS) [39] is often activated in the mobile CPU/GPU to avoid overheating by reducing the clock frequency. Since frequency reduction acts as a scaling factor on instructions, this also increases the computation time.

Accordingly, we set $\pi_1 = \frac{freq(tem^{off})}{freq(tem^{on})} \cdot [1 + f(n)]$, where $f(\cdot)$ is a linear function determined offline depicting the process switching overhead. The dynamic frequency $freq(\cdot)$ accounts for the DVFS.

- **Profiling π_2 .** We derive the memory expansion coefficient π_2 from the *Average Memory Access Time (AMAT)* principle. The effective memory latency is a weighted combination of cache and DRAM access times determined by the runtime cache hit rate φ . This is because sharing the limited cache among processes increases the memory access latency. Since cache misses incur a penalty proportional to the bandwidth gap, there is an extra latency for data movement, which is determined by the ratio between the bus bandwidth b_{DRAM} and the cache bandwidth b_{cache} (see Fig. 7). Hence, we set $\pi_2 = \varphi + (1 - \varphi) \cdot \frac{b_{DRAM}}{b_{cache}}$.

- **Profiling η_l .** Although it is challenging to measure $t_l^{c,off}$ and $t_l^{m,off}$ separately, their ratio $\eta_l = \frac{t_l^{c,off}}{t_l^{m,off}}$ can be profiled offline.

Specifically, we transform $\eta_l = \frac{c^l \cdot \delta_c}{m^l \cdot \delta_m}$, where c^l , m^l , δ_c , and δ_m denote amount of MAC, amount of memory accesses, unit MAC time, and unit memory access time of layer l , respectively. c^l and m^l can be directly derived according to the type and hyperparameters of layer l . δ_c is obtained from the processor's FLOPS F : $\delta_c = \frac{1}{F}$, while δ_m is obtained from the cache bandwidth b_{cache} : $\delta_m = \frac{1}{b_{cache}}$. We also include non-parameterized layers, e.g., activation layers, in $t_l^{c,off}$ as they also involve in gradient computation.

- **Latency Predictor Validation.** To validate the models of π_1 and π_2 , we evaluate three platforms: NVIDIA Jetson NX, Jetson Nano, and Raspberry Pi 4B. We use orthogonal micro-benchmarks to isolate coefficients by profiling computation-bound layers (large-kernel convolutions, $\eta_l \gg 1$) and memory-

bound layers (depthwise convolutions, $\eta_l \rightarrow 0$) under controlled conditions. As shown in Fig. 8(a), computation latency scales linearly across process counts ($R^2 > 0.89$), with higher temperatures shifting the curves upward and increasing the slope, consistent with thermal throttling. Fig. 8(b) demonstrates that memory latency expansion aligns with our AMAT-based model on all devices ($R^2 > 0.91$). The Jetson Nano's thermal scaling factor increases from 1.0 to 1.45, with a linear scheduling penalty (slope ≈ 0.18). The fitted DRAM-to-cache latency ratio $\frac{b_{\text{DRAM}}}{b_{\text{cache}}} = 4.9$ quantifies the cache-miss penalty.

V. ONLINE LAYER UPDATE SCHEDULER

This subsection presents a dynamic programming (DP) based lightweight online scheduler that efficiently decides the best layer update strategy $\vec{S}_e^*(N)$ for Equ.(2), given the layer-wise metrics of a DL model.

A. Problem Formulation

Picking the optimal solution to Equ.(2), an NP-hard non-linear integer programming problem, can be time-consuming [21]; We formulate specific subproblems, recursion, and space for three-stage TTA.

To apply AdaShadow across various vision architectures, such as CNNs and ViT, we categorize these networks into five unit types based on their distinct update latency patterns. We analyze the update latency rules used by the scheduler for sparse unit updates: **Convolutional/Linear layer**, **Embedding layer**, **Normalization layer**, **Multi-head attention layer**, and **Feed-forward network**. This categorization simplifies scheduling and layer updates for latency optimization, as discussed in Sec. V-C. Consider a DL model of N layers. Let the importance and update latency of layer l (starting from the last layer) be a_l^e and t_l^e , respectively, in environment e . Brute-force search for the optimal update strategy $\vec{S}_e^*(N)$ results in $O(2^N)$ complexity, which is unacceptable for online scheduling. Hence, we propose a DP formulation to solve Equ.(2). Specifically, we aim to find the optimal layer selection $\vec{S}_e^*(N)$ that maximizes the cumulative importance within the latency budget $T_{\text{bgt}} = \sigma \cdot T - T_f$. Let $P[l][t]$ be the maximum cumulative layer importance backpropagating from the last layer till layer l , given latency budget t . Further let binary vector $\vec{S}_e[l][t]$ be one solution to $P[l][t]$, and $\vec{S}_e^*[l]$ be the optimal solution. We formulate DP at the following granularity along t and l .

- *Discrete time*: We discretize time into N_T units, i.e., at a resolution of T_{bgt}/N_T . An excessively large N_T results in fine granularity, expanding the search space and decelerating the process, while a too-small N_T leads to coarse intervals and inaccurate results.
- *Layer importance threshold*: We set an importance threshold τ to select layers for updating, focusing only on those exceeding it. Layers below the threshold are skipped in the sparse update search. A higher threshold reduces complexity and latency but may lower accuracy, while a lower threshold includes more layers for better accuracy, though at a higher search cost.

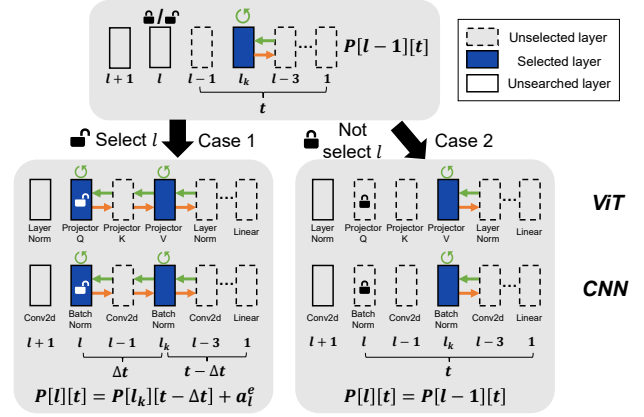


Fig. 9. Illustration of recursion of CNN and ViT in two different cases.

- *Layer-wise*: It is natural to decompose sub-problems layer-wise to support diverse DL model architectures. It also aligns with the layer-wise DL model execution scheme on mobile devices with limited resources. Finer-grained decomposition e.g., tensor-wise as [19] can be inefficient ($214 \cdot N_T$ sub-problems in tensor-wise vs. $107 \cdot N_T$ sub-problems in layer-wise for ResNet50).

Accordingly, $0 \leq l \leq N$ and $0 \leq t \leq N_T$, and the DP formulation would reduce the time complexity to $O(N^2 \cdot N_T)$ considering the time complexity $O(N)$ of solving each sub-problem (details below).

The base cases are trivial. We set $P[0][t]$ for all $0 \leq t \leq N_T$ as 0. This is because the maximum cumulative layer importance is 0 if no layer is selected for updating. The recursion for $P[l][t]$ when $l > 0$ is more subtle.

- *Case 1*: If layer l is selected, then the maximum cumulative layer importance becomes $P[l_k][t - \Delta t] + a_l^e$, where layer l_k is the last layer selected in $P[l-1][t]$, i.e., the closest selection to layer l . This is because updating one more layer decreases the latency budget when backpropagating till layer $l-1$ by Δt (details below) and increases importance by a_l^e .
- *Case 2*: If layer l is not selected, the maximum cumulative layer importance remains $P[l-1][t]$.

As shown in Fig. 9, we clarify the two cases assuming $l_k = l-2$. Since we search for the maximum, the final recursion becomes: $P[l, t] = \max\{P[l-1, t], P[l_k, t - \Delta t] + a_l^e\}$. Note that the extra latency Δt induced by selecting layer l in *Case 1* is not simply its layer update latency t_l^e . From Fig. 9, $\Delta t = t_l^{\delta w} + \sum_{m=l_k}^l t_m^{\delta x} + \sum_{m=l_k+1}^l t_m^e$, where $t_l^{\delta w}$ is the gradient calculation time of weight w_l , $\sum_{m=l_k}^{l-1} t_m^{\delta x}$ is the gradient calculation time of activations between layer $l-1$ and layer l_k , and $\sum_{m=l_k+1}^l t_m^e$ is the reforward time between layer l and layer l_k+1 . Noting that $t_l^b = t_l^{\delta x} + t_l^{\delta w}$, we utilize the amount of MAC of $t_l^{\delta x}$ and $t_l^{\delta w}$ to infer their proportion of latency in t_l^b . As l_k is unknown in advance, we gradually decrease l_k from $l-1$ to 0 to explore all possible l_k to find l_k that maximizes $P[l_k][t - \Delta t]$ with a time complexity of $O(N)$.

B. Update Strategy Search

For complex vision tasks, like ResNet50 and ViT-B/16 consist of numerous units, leading to an extensive search

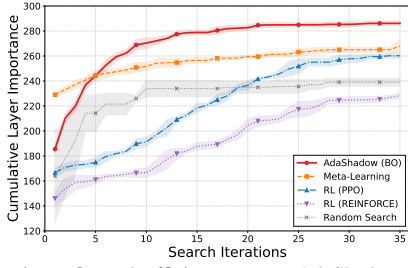


Fig. 10. Comparison of search efficiency among AdaShadow (BO), RL, Meta-Learning, and Random Search under a strict budget of 35 iterations.

space for optimal N_T and τ . Manually searching for the ideal hyperparameters N_T and τ is time-consuming and prone to error, as it critically impacts both search speed and accuracy.

To tackle this, we employ the *Bayesian Optimization (BO)* algorithm for automated selection of N_T and τ , striking a balance between search overhead and accuracy. We chose BO for AdaShadow to address mobile TTA constraints, as it efficiently locates global optima with minimal evaluations. Unlike RL, which requires thousands of interactions, or Meta-learning, which needs extensive samples for meta-training, BO excels in scenarios demanding high sample efficiency [40].

BO is a global search methodology designed to optimize multi-parameter modeling, particularly for selecting layers to maximize cumulative importance within a latency budget. Algorithm 1 shows the pseudocode of the BO search process. It begins by generating s_1 points as tuples of (N_T, τ) to initialize the candidate dataset D . It then estimates posterior distribution across the search space using this data. The acquisition function guides the selection of next search point. With input from both the posterior distribution and acquisition function, BO explores s_2 additional points, refining the search iteratively. The results update posterior distribution and acquisition function, ultimately leading to identification of the optimal point including N_T and τ from D .

The search for hyperparameters is completed before model deployment and only needs to be executed once. We set $s_1 = 10$, $s_2 = 25$, providing a large enough search space to find optimal solutions in under 1 minute. To validate efficiency, we compared AdaShadow (BO) against RL (PPO, REINFORCE), Meta-Learning, and Random Search in a micro-experiment. The goal was to find optimal scheduler hyperparameters (N_T, τ) within a 35-iteration budget ($10 s_1 + 25 s_2$). As shown in Fig. 10, BO (Red curve) converges rapidly to the optimal solution, while RL struggles in the “warm-up” phase, and Meta-Learning saturates without reaching the global optimum. These results demonstrate that BO provides the best trade-off for sample efficiency in mobile TTA. For ResNet50 on NVIDIA Jetson NX, search algorithm identifies $N_T = 500$ and $\tau = 1.2$ which can effectively balances search cost and accuracy best.

C. Unit Latency Analysis

In Sec. IV, we discussed runtime latency prediction, but the challenge lies in evaluating how dynamically adjusting layer-wise participation in backpropagation impacts overall latency. Different types of layers contribute significantly varying latencies based on whether their weights and activations are involved in gradient computations. In this subsection, we abstract their update latency into five unit types and analyze

Algorithm 1 BO search algorithm for choosing hyperparameters for sparse update strategy searching

Input: s_1 : the number of initial points; s_2 : the times of attempts to find the optimal hyperparameters;

- 1: $bayes_opt \leftarrow \text{new } bayes_opt()$ $\triangleright$ Create the BO engine
- 2: $D \leftarrow \emptyset$ $\triangleright$ Dataset of previously observed points
- 3: **for** $i = 1, 2, \dots, s_1$ **do**
- 4: $N_T \leftarrow \text{random}(0, 1000)$ $\triangleright N_T$ denotes discrete time
- 5: $b \leftarrow \text{random}(0, 10.0)$ $\triangleright \tau$ denotes layer importance threshold
- 6: $p \leftarrow (N_T, \tau)$
- 7: $y \leftarrow bayes_opt.exec(p)$ $\triangleright$ Obtain sum of searching latency and speedup rate
- 8: $D.append(p, y)$ $\triangleright$ Add the new point to D
- 9: **end for**
- 10: $bayes_opt.update(D)$ $\triangleright$ Estimate posterior distribution and acquisition function
- 11: **for** $i = 1, 2, \dots, s_2$ **do**
- 12: $p \leftarrow bayes_opt.select()$ $\triangleright$ Select the next point
- 13: $y \leftarrow bayes_opt.exec(p)$
- 14: $D.append(p, y)$
- 15: $bayes_opt.update(D)$
- 16: **end for**
- 17: **return** $bayes_opt.optimize(D)$

each based on common patterns. We breaks down latency for each layer type, focusing on reforward latency, activation gradient computation latency, and weight gradient computation latency, to guide the scheduler’s searching for optimal sparse update strategy.

CNN-ViT-Common Layers. This section evaluates the shared layers between CNNs and ViTs, *normalization layers*, with a focus on their contributions to latency during different stages of model execution. In normalization layers, the forward latency arises from the normalization process, followed by element-wise operations involving trainable scaling and shifting parameters γ and β . During backpropagation, the latency introduced by the normalization must be propagated through the gradient computation of the preceding layers. If the preceding layers contain trainable parameters, the latency incurred by the normalization operations must be integrated into the overall latency of the current layer.

- **BatchNorm Layer:** i) Forward latency (t_l^{re}) increases with batch size due to batch-level statistics. ii) Backpropagation latency ($t_l^{\delta x}$) is higher as gradients pass through batch statistics. iii) Weight update latency ($t_l^{\delta w}$) is batch-size dependent, adding complexity to updates.
- **LayerNorm Layer:** i) Forward latency (t_l^{re}) is batch-size independent, as normalization occurs at patch level. ii) Backpropagation latency ($t_l^{\delta x}$) is lower due to simpler gradient propagation. iii) Weight update latency ($t_l^{\delta w}$) depends only on the number of features, simplifying updates.

CNN-Specific Layers. This section examines the computational properties and latency of CNN-specific layers, focusing on convolutional and dense layers, both of which involve

trainable Kernel and Bias tensors. In the forward pass, the Kernel performs operations on the input, followed by bias addition. In the backward pass, gradients update the Kernel, Bias, and propagate to prior layers, forming a tensor-level computational graph that estimates training latency.

- *Convolutional Layer*: i) Forward latency t_l^{re} results from convolution and bias addition. ii) Backpropagation latency $t_l^{\delta x}$ includes gradient computation for the kernel and bias, and error propagation, scaling with spatial dimensions. iii) Weight update latency $t_l^{\delta w}$ depends on kernel size and output channels.
- *Linear Layer*: i) Forward latency t_l^{re} is based on matrix multiplication and bias addition. ii) Backpropagation latency $t_l^{\delta x}$ involves gradient computation without the spatial complexity of convolutions. iii) Weight update latency $t_l^{\delta w}$ is determined by the weight matrix size, with fewer spatial dependencies than convolutions.

ViT-Specific Layers. This section analyzes the latencies of key ViT layers, focusing on the forward pass, backpropagation, and weight updates.

- *Input & Output Embedding Layers*: i) Forward latency t_l^{re} involves lookup operations and projection to output space. ii) Backpropagation latency $t_l^{\delta x}$ is negligible for input embeddings; for output embeddings, it follows dense layer gradient propagation. iii) Weight update latency $t_l^{\delta w}$ is linked to embedding tensor updates, requiring gradient accumulation.
- *Multi-Head Attention (MHA) Layer*: i) Forward latency t_l^{re} arises from linear projections (Q, K, V) and the attention mechanism. ii) Backpropagation latency $t_l^{\delta x}$ includes gradient propagation through attention and projector weights. iii) Weight update latency $t_l^{\delta w}$ covers attention and projector weight updates.
- *Feed-Forward Network (FFN)*: i) Forward latency t_l^{re} involves two dense layers and a nonlinear activation. ii) Backpropagation latency $t_l^{\delta x}$ includes gradients through both dense layers and activation. iii) Weight update latency $t_l^{\delta w}$ pertains to parameter updates in both layers, independent of the activation function.

D. Reducing the Scheduler Cost

Given the above formulations, we can explore all $P[l][t]$ recursively till $P[N][T_{bgt}]$, and then we can search the optimal layer update strategy $\tilde{S}_e^*[N]$.

To further improve the efficiency, we refine the search space by removing two types of invalid sub-problems.

- For T_{bgt} , we iteratively exclude sub-problem $P[l, t]$ if t exceeds the latency budget T_{bgt} . We can also discard all sub-problems generated from an invalid $P[l, t]$.
- For t in sub-problem $P[l, t]$, if the gradient computation time $\sum_{m=1}^l t_{\delta x}^m$ exceeds t , the total latency of $P[l, t]$ will also exceed t even if no layers are selected for updating. Hence, we can discard such $P[l, t]$ and all sub-problems it generates.

Moreover, to eliminate redundant calculations, we identify overlapping solutions by formulating the latency of both

backward and reforward into the same recursive sub-problem and formula. This is because, as highlighted in § II-C, we observe that the *position of the first update layer* impacts the latency of both backward and reforward simultaneously.

VI. IMPLEMENTATION

This subsection presents how the layer importance assessor (§ III), layer latency predictor (§ IV), and layer update scheduler (§ V) cooperates in the *forward-backward-reforward* pipeline (see Fig. 4) as well as our *additional optimizations* to improve the accuracy and efficiency of TTA on mobiles.

At initialization, DL models are pre-trained in the cloud using source datasets and then deployed to mobile devices. Upon receiving a batch of test samples x_e from a new environment e , AdaShadow adapts the DL model as follows.

- *Forward phase.* AdaShadow employs the layer importance assessor and the latency predictor, which profiles runtime coefficients (π_1, π_2) for each batch to capture rapid resource fluctuations, to derive the layer importance a_l^e and the backward and reforward latency t_l^b and t_l^{re} for each layer of the DL model. Meanwhile, it prepares each layer's output feature maps o_l^e and computation graph for the backward phase.
- *Backward phase.* AdaShadow calls the layer update scheduler to search for the optimal strategy $\tilde{S}_e^*(N)$, based on a_l^e , t_l^b , and t_l^{re} obtained in the forward stage. AdaShadow prunes computation graph nodes of layers not in $\tilde{S}_e^*(N)$ to skip gradient calculation at the compiler level, to realize the selective propagation process. The training loss will be discussed later.
- *Reforward phase.* AdaShadow infers on x_e using the updated model from the backward stage. AdaShadow partially reuses computation results from the forward stage to accelerate the inference, as described next.

We propose two more optimizations during backward and reforward to boost the performance.

- *Adaptation loss for small batches.* Existing TTA methods [10], [41] mainly use the *entropy* of the *final layer* output as the unsupervised adaptation loss. However, they often suffer from low accuracy when adapting to small batches [41]. In AdaShadow, we refine the loss L by summing up the KL-divergence between the new environment and the history from *all layers*, i.e., $L = \sum_{l=1}^N D_{KL}(E_l^H \| E_l^e)$. Evaluations show that the refined loss is more robust to small batch sizes.

- *Memory I/O-aware computation reuse between forward and reforward.* As both forward and reforward are performed on the same batch, we can partially reuse the computations from the forward stage to further reduce the overall latency, as shown in Fig. 11. Specifically, we track the memory addresses for input tensors during forward. After determining the layer update strategy, we detach the input activation x_n of the first layer n in $\tilde{S}_e^*(N)$ from the gradient computation graph and keep it in memory, while releasing the input tensors x_i (where $i \neq n$) for other layers. Before reforward begins, we directly load the input tensor x_n from memory and reuse computation results for layers that were not updated during the backward pass, starting from the first updated layer in the model. This

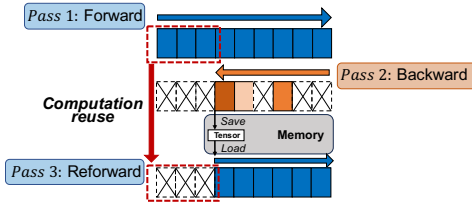


Fig. 11. Illustration of memory I/O-aware computation reuse implementation.

tensor-wise computation reuse skips layers before the first updated layer n during the reforward phase.

This method optimizes both CNNs and ViTs by reducing redundant computation and memory usage.

- In **CNN**, it targets early convolutional layers, retaining input activations and eliminating recomputation for layers unaffected by gradient updates. This reduces memory I/O and computational latency.
- In **ViT**, the method significantly cuts latency by reusing intermediate activations in self-attention and feed-forward layers, which involve heavy matrix operations.

VII. EVALUATION

A. Experiment Setup

Implementation. We prototype AdaShadow on three mobile devices. D_1 : NVIDIA Jetson NX (Cortex-A57 CPU and 256 CUDA cores GPU); D_2 : NVIDIA Jetson Nano (Cortex-A57 CPU and 128 CUDA core GPU); D_3 : Raspberry Pi 4B (Cortex-A72 CPU). We installed Jetpack 4.6.4 OS based on Ubuntu 18.04 on D_1 and D_2 , and the 64-bit Raspbian 11 OS on D_3 . Before DL model deployment, AdaShadow measures the multi-process scheduling latency function $f(n)$ and the offline layer execution latency via one pass of backpropagation using PyTorch profiler plugin, and detects frequency-temperature function $freq(tem)$ through the device DVFS strategy. During test-time adaptation, AdaShadow uses the SGD optimizer with a learning rate lr of $5e-3$.

Tasks, Datasets, and Models. We test three types of mobile sensing tasks across CNN and ViT models, with diverse continual domain shifts.

- **NICO++** [42]. It covers 6 types of *environmental* shifts, including outdoor, autumn, dim, grass, rock, and water. The model is pre-trained on 'outdoor' for adaptation to the other five environments. NICO++ are commonly used datasets for evaluating domain adaptation.
- **CIFAR10-C and CIFAR100-C** [43]. CIFAR10-C and CIFAR100-C has been widely used for evaluation adaptation performance in mobile systems. They simulate 15 types of *corruption* shifts, induced by lens shakes, low-light conditions, and sensor degradation such as Gaussian noise, motion blur, pixelation, *etc.* We experiment with two dataset combinations: [CIFAR10, CIFAR10-C] and [CIFAR100, CIFAR100-C], where the first is for pre-training and the second for adaptation.
- **Self-collected mobile videos.** It contains 10 natural, continual, potentially *mixed* shifts collected in open mobile environments. Specifically, we built an Ackermann vehicle platform equipped with Jetson NX development

board, STM32 control board, visual sensors, *etc.* to move on campus and collect 10 hours of videos. We manually sample video clips that contain transitions of shifts, *e.g.*, indoor-wild, sunny-rainy-cloudy, daytime-night, and different view angles, *etc.*

- **Self-collected layer adaptation latency records.** We build a dataset of 15,000 records capturing various real-world system factors and layer adaptation latency.

We evaluate eight typical models architectures:

- **CNN Architectures:** We use VGG16 (no BN) [44], MobileNetV2 (lightweight, with BN) [45], and ResNet50 (standard, with BN) [35].
- **ViT Architectures:** We include ViT-B/16 [25], DeiT-tiny [46], and MobileViTv3-XS [47], with additional testing on larger models like ViT-L/16 [25] and Swin-B [48]. To optimize for mobile deployment, we use the PyTorch Image Models (`timm`) library, initialized with ImageNet-1k weights. To address memory bandwidth limits on edge devices, we truncate the input sequence to 512 tokens, balancing global attention context with latency constraints.

Baselines. Generic and supervised TTA set strict accuracy lines, while efficient TTA set latency benchmarks.

- **Generic TTA (no target label).** They aim at accurate TTA without explicit efficiency optimization.
 - **Tent** [10]: the first TTA that updates the *BN* layers via entropy minimization.
 - **CoTTA** [11]: another representative TTA strategy via *knowledge distillation* for anti-forgetting.
- **Efficient TTA (no target label).** Besides accuracy, this category optimizes efficiency metrics of TTA.
 - **EATA** [13]: *data-efficient* TTA by updating BN layers on selective samples.
 - **EcoTTA** [14]: typical *memory-efficient* TTA that reduces the memory footprint to update the BN layers with lightweight adapters.
 - **MECTA** [12]: *memory-efficient* TTA that selectively updates the proposed *adaptive BN* layers.
 - **BFTT3D** [16]: *backpropagation-free* TTA via comparing source and target features.
 - **LAME** [49]: a *latency-efficient* TTA that adapts the model's output rather than parameters using manifold-regularized likelihood loss.
 - **OFTTA** [17]: a *latency-efficient* TTA that selectively adjust the proposed *exponential decay BN* layers.
- **Efficient Training (supervised by target labels).** This category focuses on efficient training on low-resource mobile and embedded devices. They provide unsupervised loss (*i.e.*, entropy) at test time.
 - **Melon** [24]: a *memory-efficient* on-device training scheme that uses recomputation and micro-batch to fit into memory-scarce mobile devices.
 - **TTE** [21]: a *memory-efficient* on-device training scheme that selectively updates tensors via *offline* profiling.
 - **ElasticTrainer** [19]: a *latency-elastic* on-device training method that uses *runtime* profiling to iden-

TABLE III
TEST ACCURACY (%), ADAPTATION THROUGHPUT (FPS), AND LATENCY (MS) OF RESNET50 WITH CONTINUAL ENVIRONMENTAL SHIFTS.

Time		Continual data shift over time $\rightarrow$										Mean ACC.(%)	Th.(fps)	Latency(ms)
Dataset	Method	Autumn	Dim	Grass	Rock	Water	Autumn	Dim	Grass	Rock	Water			
NICO++	TENT [10]	83.5	82.0	86.4	85.7	86.0	89.5	89.8	90.2	90.3	91.4	87.5	30.5	32.8
	CoTTA [11]	84.7	82.1	87.0	86.9	85.4	90.1	89.4	92.8	90.8	91.5	88.1	4.1	243.9
	EATA(C) [13]	80.3	79.8	84.5	84.2	82.8	86.3	86.5	87.4	88.8	86.1	84.7	35.5	28.2
	EATA(O) [13]	84.9	82.7	87.2	86.4	86.7	90.8	89.2	92.6	92.0	91.5	88.4	27.1	36.9
	EcoTTA(C) [14]	84.2	81.3	85.2	85.1	83.9	88.5	86.3	90.7	89.1	89.2	86.4	31.8	31.4
	EcoTTA(O) [14]	85.9	85.0	88.8	86.2	86.3	91.1	90.4	92.5	91.7	92.0	89.0	23.6	42.4
	MECTA [12]	86.0	83.0	88.9	87.7	86.5	92.5	90.5	90.2	91.5	92.5	88.9	31.4	31.8
	Melon [24]	73.5	51.0	46.1	40.8	25.6	23.0	14.0	9.0	8.6	6.0	29.8	22.4	44.6
	TTE [21]	67.5	66.1	71.0	70.3	69.0	74.0	72.0	75.3	73.5	75.9	71.5	29.0	34.5
	ElasticTrainer [19]	73.0	69.2	74.5	74.5	73.1	76.7	77.3	78.1	78.4	79.0	75.4	57.0	17.5
	AdaShadow	86.2	84.5	89.2	87.9	87.0	91.8	90.6	93.4	92.7	93.1	89.6	76.0	13.2

TABLE IV
TEST ACCURACY (%), ADAPTATION THROUGHPUT (FPS), AND LATENCY (MS) OF RESNET50 WITH CONTINUAL CORRUPTION SHIFTS.

Time		t $\rightarrow$														Mean ACC.(%)	Th.(fps)	Latency(ms)
Dataset	Method	Gaus.	Shot	Impu.	Defo.	Glas.	Moti.	Zoom	Snow	Fros.	Fog	Brig.	Cont.	Elas.	Pixe.	Jpeg		
CIFAR-10C	TENT [10]	72.8	73.8	67.7	80.2	68.0	80.9	84.5	77.2	86.2	85.8	81.2	88.5	70.2	77.7	77.4	78.1	28.3
	EATA [13]	68.5	72.0	71.4	84.2	71.5	81.0	85.4	80.4	83.9	85.6	86.3	87.3	75.3	83.5	73.4	79.3	27.3
	EcoTTA [14]	66.1	69.6	70.4	78.7	68.5	79.4	85.5	76.6	86.6	88.5	90.8	82.0	72.7	77.5	73.5	77.8	23.0
	MECTA [12]	73.1	70.2	69.7	83.1	69.0	82.6	83.8	79.9	84.7	89.3	91.3	85.0	75.5	86.0	72.5	79.7	30.6
	BFTT3D [16]	43.7	46.1	51.9	62.2	40.8	54.0	66.6	55.7	60.8	64.8	64.9	57.1	52.2	59.4	46.9	55.1	163.2
	LAME [49]	48.7	44.2	50.4	59.8	44.4	55.7	66.9	55.8	61.5	58.3	62.9	68.5	54.7	61.6	44.8	55.9	176.3
	OFTTA [17]	47.9	46.0	46.1	57.5	46.5	64.8	62.8	55.3	58.3	67.1	67.4	66.6	54.8	61.8	50.0	56.9	191.6
	ElasticTrainer [19]	53.9	59.5	59.1	72.3	58.6	71.3	74.1	69.1	71.1	68.6	73.1	75.1	60.7	66.3	60.9	66.2	58.0
	AdaShadow	69.7	74.7	71.9	84.3	71.9	80.3	85.7	84.4	86.7	87.3	91.6	84.3	75.7	86.2	74.5	80.6	78.6

TABLE V
PERFORMANCE COMPARISON OF RESNET50 ON CIFAR100-C.

Dataset	Time	Method	Mean ACC.(%)	Th.(fps)	Latency(ms)
CIFAR-100C		TENT [10]	60.6	25.9	38.6
		EATA [13]	58.1	23.5	42.6
		EcoTTA [14]	58.5	20.5	48.8
		MECTA [12]	61.5	26.9	37.2
		BFTT3D [16]	43.4	159.4	6.3
		LAME [49]	42.2	164.0	6.1
		OFTTA [17]	43.0	174.6	5.7
		ElasticTrainer [19]	49.6	56.2	17.8
		AdaShadow	62.7	75.4	13.3

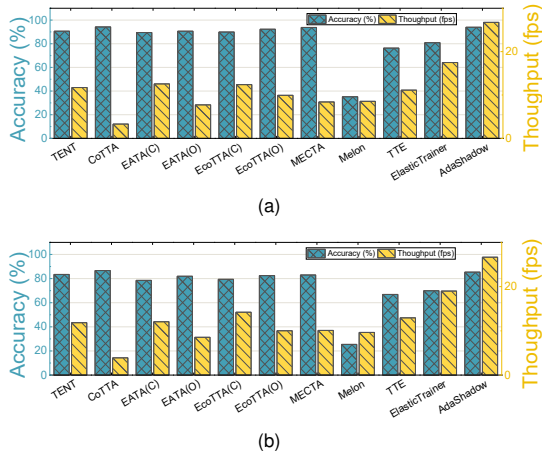


Fig. 12. Performance comparison of ViT-B/16 with continual (a) environmental shifts and (b) corruption shifts.

tify important tensors and *offline* latency profiling, with the *backpropagation*-based importance metric.

EATA(P) and EcoTTA(P) denote the EATA and EcoTTA baselines which are implemented in parallel adaptation/inference mode. Other baselines are in sequential adaptation/inference mode by default, only applying the newly adapted model to subsequent sample inference for accuracy testing (as discussed in § II-B). We test *i)* the inference accuracy using newly adapted models with test data, *ii)* the sum latency of forward, backward, and reforward stages, and *iii)* the inference throughput with a certain data streaming rate.

B. Main Results

1) *Accuracy vs. Latency on ResNet50 with Continual Environment Shifts:* This experiment evaluates the performance when adapting to two types of *continual shifts* on ResNet50. We pre-train a ResNet50 with the “outdoor” sub-dataset in NICO++ and sequentially adapt it to five other sub-datasets with *environmental shifts* (i.e., autumn, dim, grass, rock, and water) on D_1 (see Tab. III). AdaShadow balances adaptation accuracy and latency better than baselines, achieving $2.1\times \sim 3.2\times$ lower latency than Tent, EATA, and EcoTTA, and up to $18.5\times$ lower than CoTTA. Although ElasticTrainer accelerates other baselines by $2\times$, the latency of AdaShadow is only 76.9% that of ElasticTrainer. Moreover, we observe that AdaShadow is even faster than existing TTA methods in parallel inference/adaptation mode. AdaShadow also achieves the highest adaptation accuracy. Compared to generic and efficient TTA baselines, AdaShadow exhibits up to 5.0% higher accuracy, enhancing learning capabilities of fixed up-date substructures. MECTA has comparable accuracy due to its adaptive BN layers. As for efficient training baselines, in label-free environments, AdaShadow outperforms Melon by up to 59.9%, mitigating overfitting with sparse updating. Moreover, AdaShadow shows an accuracy improvement of 18.2% and 14.3% over TTE and ElasticTrainer, respectively, due to its backpropagation-free and timely layer importance assessor.

2) *Accuracy vs. Latency on ViT-B/16 with Continual Environment Shifts:* This experiment evaluates the performance of adapting ViT-B/16 to *continual environment shifts* on NICO++, following the experimental settings outlined in VII-B1. AdaShadow outperforms baselines in both adaptation accuracy and latency, achieving latency reductions of $2.1\times$ to $3.4\times$ compared to Tent, EATA, EcoTTA, and MECTA, and up to $8.1\times$ lower than CoTTA. It operates at only 65.4% of ElasticTrainer’s latency while achieving the highest adaptation accuracy in parallel inference/adaptation mode. Compared to generic TTA baselines, AdaShadow demonstrates up to 4.5% higher accuracy, with MECTA’s adaptive normalization layers offering comparable performance. In label-free environments,

AdaShadow surpasses Melon by 58.8% and improves accuracy by 17.7% and 13.1% over TTE and ElasticTrainer, respectively, due to its backpropagation-free design and timely layer importance assessment.

3) *Accuracy vs. Latency on ResNet with Continual Corruption Shifts*: We further compare AdaShadow with baselines in scenarios with *corruption* shifts, another common shift type in mobile applications caused by sensor degradation. AdaShadow still achieves the best accuracy-latency balance when continually adapting to the corrupted CIFAR10 (see Tab. IV and CIFAR100 (see Tab. V) datasets on device D_1 . In CIFAR10-C shifts, AdaShadow outperforms EcoTTA, MECTA, and ElasticTrainer, the three most efficient baselines, achieving accuracy improvements of 2.9%, 0.9% and 14.4%, and acceleration of $3.4\times$, $2.6\times$ and $1.2\times$, respectively. For CIFAR100-C, AdaShadow surpasses EcoTTA, MECTA and ElasticTrainer with accuracy gains of 4.2%, 1.2% and 13.1%, and acceleration of $3.7\times$, $2.8\times$ and $1.3\times$. We also compare AdaShadow with backward-free TTA methods, BFTT3D, LAME, and OFTTA, on CIFAR10-C and CIFAR100-C (see Tab. IV). Although AdaShadow is on average 2.3 times slower, backward-free methods experience a 23.8%-25.4% accuracy drop after more than two continual shifts, which is unacceptable for mobile tasks. These methods achieve acceptable accuracy in single shift scenarios but lack adaptability in continual shift scenarios and significant image changes because prohibiting backward and parameter updating limit the model's optimization capacity [50]. OFTTA and BFTT3D mitigate prior distribution shifts but offer little improvement for appearance shifts. LAME even performs worse than the model without adaptation in scenarios with sensor degradation such as *Shot*, *Gaus.*, and *Pixe*.

4) *Accuracy vs. Latency on ViT-B/16 with Continual Corruption Shifts*: This experiment evaluates the performance of adapting ViT-B/16 to *continual corruption shifts* on NICO++, following the experimental settings outlined in VII-B3. AdaShadow outperforms EcoTTA, MECTA, and Tent, achieving accuracy improvements of 5.9%, 2.4%, and 1.9%, with acceleration factors of $1.9\times$, $2.6\times$, and $2.3\times$. It operates at only 71.4% of ElasticTrainer's latency while achieving the highest adaptation accuracy in parallel inference/adaptation mode. Compared to generic TTA baselines, AdaShadow shows up to 6.9% higher accuracy, with MECTA's adaptive normalization layers providing comparable performance. In label-free environments, AdaShadow increases accuracy by 18.6% and 15.5% over TTE and ElasticTrainer, respectively, while surpassing Melon by 59.9%. This improvement is attributed to its backpropagation-free design and efficient layer importance assessment.

In summary, AdaShadow outperforms other baselines in adaptation latency and accuracy across diverse non-stationary shifts for both ResNet50 and ViT-B/16 due to its efficient sparse updating with accurate online predictors for unit update importance and latency.

C. Performance across Diverse Mobile Devices

This experiment verifies AdaShadow's effectiveness under different hardware and DVFS strategies by comparing to

TABLE VI
ADAPTATION LATENCY ON DIVERSE DEVICES.

Diverse devices	NICO++			CIFAR10-C		
	D_1	D_2	D_3	D_1	D_2	D_3
EATA [13]	39.5	318.5	1960.8	36.4	344.8	2325.6
EcoTTA [14]	44.1	322.6	2234.7	43.1	405.2	3031.8
ElasticTrainer [19]	16.4	144.9	831.6	16.2	136.6	1013.2
Ours	12.9	113.6	769.2	12.5	121.9	909.1

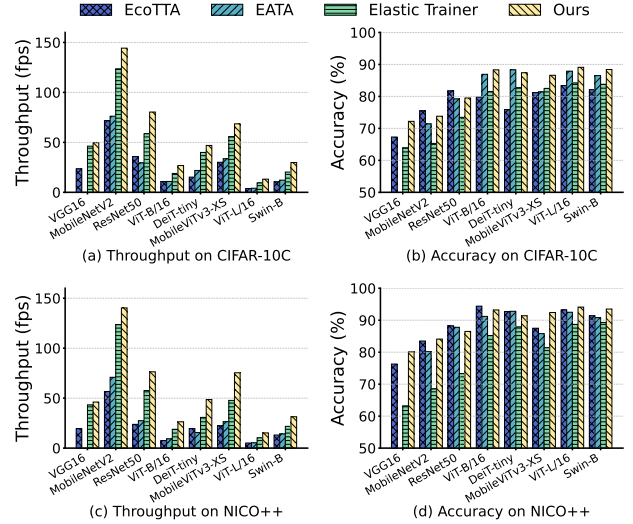


Fig. 13. Adaptation performance with diverse DL models.

EATA, EcoTTA, and ElasticTrainer on three devices (*i.e.*, $D_1 \sim D_3$) using NICO++ and CIFAR10-C shifted data. As shown in Tab. VI, AdaShadow achieves the lowest adaptation latency, being at most 32.7%, 29.3%, and 77.1% that of EATA, EcoTTA, and ElasticTrainer, respectively. The latency difference between AdaShadow and ElasticTrainer is smaller on device D_3 due to increased GPU core parallelization, increasing differences between runtime and offline latency, highlighting AdaShadow's advantage.

D. Scalability over Diverse Model Architectures

We compare AdaShadow's adaptation accuracy and throughput with EATA, EcoTTA, and ElasticTrainer on VGG16, MobileNetV2, and ResNet50, ViT-B/16, DeiT-tiny, MobileViTv3-XS, ViT-L/16 and Swin-B using NICO++ and CIFAR10-C datasets on device D_1 . From Fig. 13, AdaShadow achieves $2\times \sim 3\times$ acceleration while maintaining high accuracy across all models. ElasticTrainer's backpropagation-dependent profiler leads to poor unsupervised accuracy, while MobileNetV2's simplified structure limits learnability and slows convergence. Furthermore, fixed sparse updating in EcoTTA and EATA proves insufficient for lightweight models. AdaShadow addresses these limitations through flexible layer selection, bridging accuracy gaps. Its unit-based latency predictor further ensures an optimal accuracy-latency tradeoff, particularly on ViT-B/16. In contrast, models like DeiT-tiny and MobileViTv3-XS sacrifice accuracy and flexibility in dynamic environments due to fixed sparsity. For large-scale architectures such as ViT-L/16 and Swin-B, AdaShadow demonstrates superior scalability. By identifying and updating only critical layers, our method achieves higher throughput than baselines while maintaining competitive accuracy despite high computational costs.

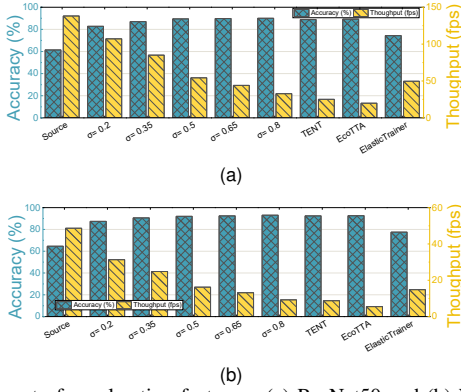


Fig. 14. Impact of acceleration factor on (a) ResNet50 and (b) ViT-B/16.

TABLE VII
IMPACT OF BATCH SIZES (BS).

	BS=1	BS=2	BS=4	BS=16
Accuracy (%)	79.5	84.5	88.5	89.7
Throughput (fps)	19.2	43.2	79.6	340.2
Peak memory (MB)	2108	2210	2462	3794

E. Micro-benchmarks

1) *Impact of Expected Acceleration Factor σ* : To analyze the trade-off between sparse updating latency and accuracy, we vary the acceleration factor σ in Eq. (2). Lower σ updates fewer, more critical layers, while higher σ updates more layers. As shown in Fig. 14, **AdaShadow** maintains near-optimal accuracy (above 85% for ResNet50 and 90% for ViT) across a wide range of σ values (0.2 to 0.8), confirming the effectiveness of our *Backpropagation-free Importance Assessor*. Diminishing returns occur when σ exceeds 0.5, where accuracy saturates and throughput declines. Accuracy drops significantly below $\sigma < 0.2$, marking the minimum sparsity for capturing domain shifts. We set $\sigma = 0.33$ as the default, balancing accuracy and throughput with $2\times$ to $3\times$ speedup compared to baselines for real-time mobile applications.

2) *Impact of Adaptation Rate*: We test the **AdaShadow**'s sensitivity to adaptation rate using CIFAR10-C on ResNet50 and ViT-B/16, including integration rate α and learning rate lr . **AdaShadow** shows robustness to lr variations (see Fig. 15b) but is sensitive to α . A small α leads to the model hardly learning new scenes, while a large α causes the model to easily overfit to new scenes (see Fig. 15a). Therefore, we empirically set $\alpha = 0.1$ in **AdaShadow** and $lr=5e-3$ by default.

3) *Impact of Batch Sizes*: For stay responsive to non-stationary environments, effective adaptation on small batch sizes is expected. Tab. VII shows the performance of **AdaShadow** with batch sizes of 1, 2, 4, and 16 with NICO++ on device D_1 . Even a batch size of 1 achieves high accuracy, e.g., 79.5%, because layer importance assessor efficiently estimates distributions in small-batch mobile data using only first and second-order moments through channel-wise extraction. As the batch size increases, throughput and accuracy improve at the cost of memory usage. With a batch size of 16, it surpasses the device's memory budget. Thus, we set *batch size* = 4 by default in **AdaShadow**.

4) *Generalizing to Two-stage Adaptation Mode*: This experiment tests the **AdaShadow**'s efficiency of different adaptation modes using NICO++ and ResNet50. We compare

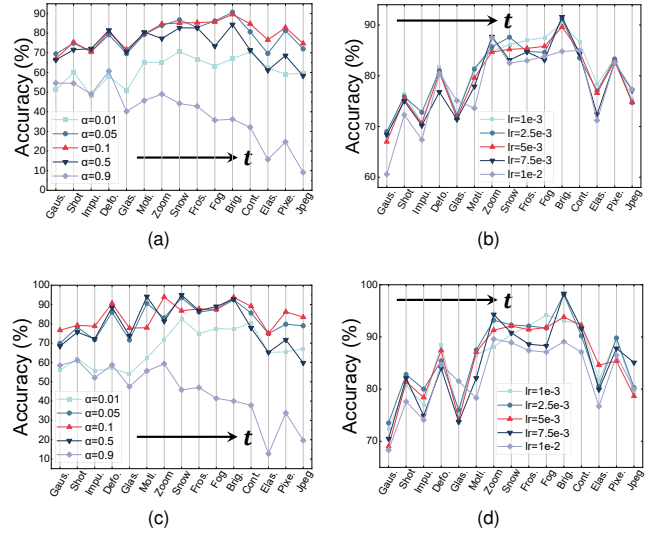
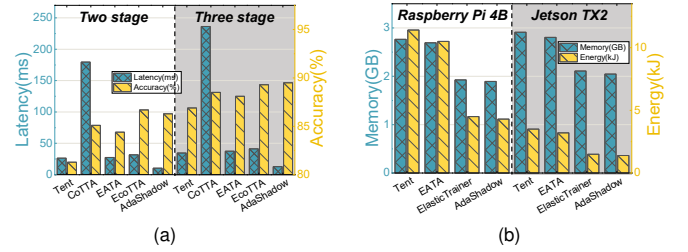
Fig. 15. Impact of (a) σ , (b) lr , (c) ViT σ , and (d) ViT lr .

Fig. 16. Evaluation of (a) different adaptation modes and (b) system overhead.

the effectiveness of **AdaShadow** in three-stage (forward-backward-reforward) and two-stage (forward-backward) on-line TTA settings (Fig. 16a). Results show that **AdaShadow** also aligns well with the traditional two-stage (forward-backward) mode, achieving significant acceleration by optimizing the sparse updating in the backward pass.

5) *Performance of Latency Predictor*: We test **AdaShadow**'s latency predictor under offline and $dr_1 \sim dr_4$ conditions (the same settings mentioned in § IV) using 15,000 self-collected records from different systems, devices, and models. Considering dynamic kernel and memory resources, the average prediction error between **AdaShadow**'s latency predictor and ground truth under $dr_1 \sim dr_4$ are $\leq 3.3\%$ (see Tab. VIII), showing high accuracy and stability.

6) *Sensitivity to Search Hyperparameters*: We evaluate **AdaShadow**'s sensitivity to time resolution N_T and importance threshold τ by analyzing cumulative importance and search latency across $N_T \in [100, 1000]$ and $\tau \in [0.1, 2.0]$. As shown in Fig. 18, the system remains stable, with performance saturating at $N_T \approx 500$ and staying robust for $\tau \leq 1.2$. Since search overhead increases with N_T but decreases with higher τ , our configuration ($N_T = 500, \tau = 1.2$) balances maximum cumulative importance with strict latency constraints.

7) *System Overhead*: **AdaShadow** can efficiently tune memory usage by selective updates of crucial layers in a dynamic adaptive manner. In Fig. 16b, **AdaShadow** achieves a peak memory usage of $\leq 2045MB$, lower than Tent, EATA, and ElasticTrainer. Moreover, **AdaShadow** significantly decreases the frequency and duration of backpropagation, reducing 56% and 60% energy cost than Tent and EATA.

TABLE VIII
ERROR OF FACTORS π_1 AND π_2 .

	Offline	dr_1	dr_2	dr_3	dr_4
π_1	1.0	1.6	4.3	1.0	7.0
π_2	1.0	1.0	1.0	2.4	2.4
Offline latency	45.8	45.8	45.8	45.8	45.8
Actual latency	45.3	73.5	187.2	50.5	309.9
Predicted latency	45.8	71.7	181.7	52.2	299.8
Error rate (%)	1.2	2.4	2.9	3.3	3.2

TABLE IX
PERFORMANCE COMPARISON IN SEQUENTIAL ADAPTATION/INFERENCE MODE.

	Accuracy (%)	Throughput (fps)	Turnaround time
EATA [13]	70.8	35.3	2.11
EcoTTA [14]	71.4	22.8	2.42
ElasticTrainer [19]	60.7	70.6	1.47
AdaShadow	76.5	80.8	1.37

We also profile AdaShadow's runtime components using ResNet50 on an NVIDIA Jetson NX. As shown in Fig. 17, AdaShadow reduces latency from 32.8 ms (Full Update) to 14.1 ms, with a minimal control overhead of ~ 1.2 ms. The *Importance Assessor* takes 0.12 ms (8.6 KB), the *Latency Predictor* incurs 0.03 ms (0.4 KB), and the *Update Scheduler* completes in 1.05 ms (132 KB) due to efficient pruning. These results confirm that the overhead is minimal and justified by the performance gains.

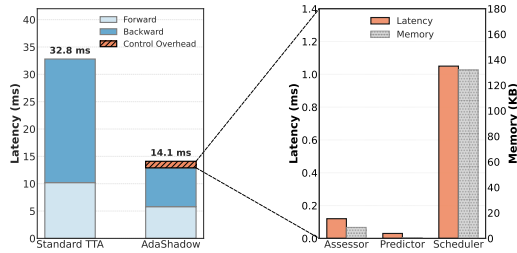


Fig. 17. Runtime component overhead of AdaShadow on NVIDIA Jetson NX using ResNet50 dataset.

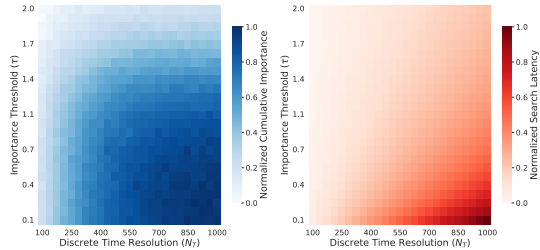


Fig. 18. Sensitivity analysis of hyperparameters N_T and τ . (a) Normalized cumulative importance. (b) Normalized search latency.

F. Real-world Case Studies

1) *Sequential Adaptation/Inference Mode*: This study showcases the usage of AdaShadow, where it first adapts the model upon a new batch of data before inference on the same batch using the updated model (see Fig. 19a). That is, the inference on *batch1* is after the adaptation on *batch1*, and the adaption on *batch2* also starts after the adaptation on *batch1*. This execution mode slightly prioritizes accuracy over latency and is widely used in applications such as AR and VR. We set the time window to 15ms, the batch size to 4, and test with continuously played CIFAR10-C on device D_1 . As shown in Fig. 19a, this execution mode introduces waiting time, making latency a potential bottleneck. We quantify such waiting time via the *turnaround time* for batch i , i.e., $r^i = \frac{T_{wait}^i + T_a^i + T_{re}^i}{T_a^i + T_{re}^i}$, a common metric in OS process scheduling, where T_{wait}^i

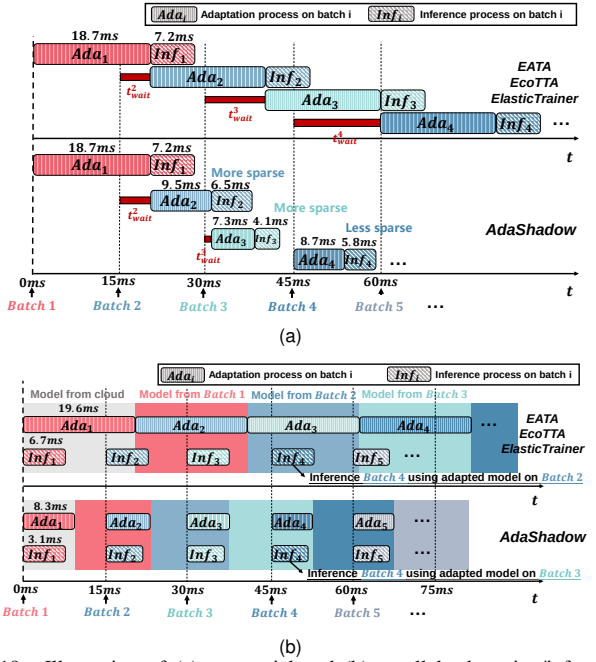


Fig. 19. Illustration of (a) sequential and (b) parallel adaptation/inference mode on ResNet50.

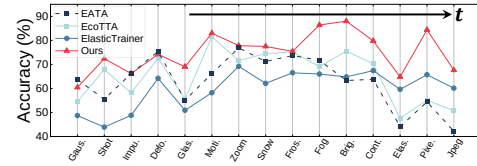


Fig. 20. Accuracy comparison in parallel adaptation/inference mode.

is the wait time of batch i . We then report the average r across all arriving data batches as the total waiting time. Since AdaShadow dynamically adjusts the acceleration factor σ , it can gradually decrease r , where AdaShadow scales σ to tune t_{wait}^2 , t_{wait}^3 , and t_{wait}^i of subsequent arriving batch. From Tab. IX, AdaShadow achieves higher inference accuracy with a lower r than the baselines.

2) *Parallel Adaptation/Inference Mode*: This study demonstrates the usage of AdaShadow, where when a new batch of data arrives, it directly utilizes the current model for inference without waiting for the newly adapted one (see Fig. 19b). This execution mode is preferred in real-time applications like drone object detection and autonomous driving. We set the time window to 15ms, the batch size to 4, and test with CIFAR10-C on device D_1 . From Fig. 20, AdaShadow maintains the highest accuracy for its rapid and accurate model adaptation. In contrast, other schemes fail to update model in time, leading to inaccurate inference using outdated models.

3) *Use Case*: This study evaluates AdaShadow in real-world mobile environments. We built a vehicle platform to continuously collect video streams in a university campus in varying weather, illumination, and locations. We performed test-time DL model adaptation and inference during vehicle driving. We summarized the environments encountered over time as: bright indoor (e_1) $\rightarrow$ rainy road (e_2) $\rightarrow$ uphill $\rightarrow$ rainy grass (e_3) $\rightarrow$ downhill $\rightarrow$ night road (e_4) $\rightarrow$ dark indoor (e_5). From Fig. 21, AdaShadow can promptly adapt to all these natural shifts with high accuracy and low latency by updating diverse layers. The blue bars below represent the

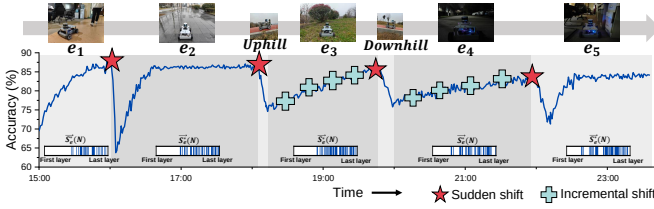


Fig. 21. Case study illustration. $\bar{S}_e^*(N)$ is the corresponding optimal sparse updating strategy.

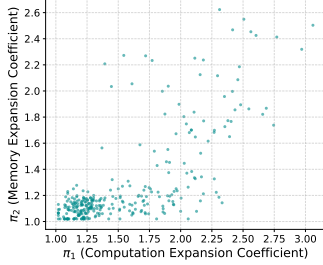


Fig. 22. Distribution of runtime computation (π_1) and memory (π_2) expansion coefficients collected from 300 real-world batches.

layers that are selected for updating.

We further analyzed 300 consecutive data batches from the case study and visualized the distribution of runtime computation (π_1) and memory (π_2) expansion coefficients in Fig. 22. The results show common offline-to-runtime latency deviations. Specifically, 47% of batches have moderate latency increases due to warm-up and caching ($\pi_1 \approx 1.2, \pi_2 \approx 1.1$), while 53% exhibit larger deviations: 28% from thermal throttling ($\pi_1 \approx 1.7$) and 25% from system contention, highlighting the need for the layer latency predictor.

VIII. RELATED WORK

On-device CNN Training. Resource-efficient on-device training often employs recomputation [24], micro-batching [51], and memory swapping [52] to mitigate memory constraints. However, these methods frequently increase latency due to computational or data-transfer overhead. In contrast, sparse updating [19], [21] accelerates training by reducing backpropagation workloads.

Sparse updating can be *static* [53], involving fixed modules, or *dynamic* [12], [19], where modules are selected at runtime. Static strategies are ill-suited for Test-Time Adaptation (TTA) as critical layers vary across domains. While AdaShadow shares the dynamic update philosophy of ElasticTrainer [19], the latter is limited by its reliance on labeled data and offline training assumptions. ElasticTrainer fails to account for the tight latency budgets and fluctuating resource dynamics inherent in TTA. AdaShadow addresses these challenges, providing accurate, low-latency adaptation for unlabeled data in highly dynamic mobile environments.

On-device ViT Fine-tuning. As ViTs scale, parameter-efficient fine-tuning (PEFT) becomes essential to mitigate the high resource demands of downstream adaptation while preventing forgetting and overfitting. PEFT for ViTs typically falls into three categories: (i) *Partial Tuning* [22], [54] updates only specific internal parameters; for instance, [54] tunes attention weights for resolution adaptation, while GreenTrainer

[22] adaptively selects key parameters to minimize FLOPs. (ii) *Prompt Tuning* [55] prepends learnable tokens to the input, such as VoP [55], which achieves efficient text-video retrieval with minimal overhead. (iii) *Adapter Tuning* [56] inserts lightweight modules into the backbone, with methods like VMT-Adapter [56] enabling scalable multi-task adaptation. Despite the high accuracy of *Partial Tuning*, its rigid update patterns often falter in non-stationary environments. In contrast, *Prompt* and *Adapter Tuning* offer superior flexibility for test-time adaptation (TTA). AdaShadow leverages this by dynamically adjusting sparse update strategies according to the environment and seamlessly integrating with both prompt and adapter-based methods to maximize adaptation efficiency.

Test-time Adaptation. Test-time adaptation (TTA) is an emerging *domain adaptation* setting with *unlabeled* target data and no access to source data or supervision [10]. Generic domain adaptation techniques [57] are inherently supervised. More recent unsupervised domain adaptation methods [58] still rely on source data to assess and align data distributions. A comprehensive review on domain adaptation can be found in [9]. Earlier TTA efforts focused on improving adaptation *accuracy* in this unsupervised, source-free setup. Classical methods [10] adapt the *batch normalization (BN)* layers by *minimizing the entropy of the last layer's outputs on both training and testing data*. Recent studies [11], [14] show that updating BN layers alone may be insufficient for more drastic shifts, especially with small batches, and propose updating parameters in convolutional layers [11] or adapters [14]. In our approach, we update all layers rather than restrict updates to BN layers and refine the adaptation loss by incorporating outputs from all layers to improve robustness to small batches.

A few studies have explored improving the *efficiency* of TTA. EATA [13] focuses on data efficiency by adapting only important test samples. EcoTTA [14] uses lightweight meta-networks for memory-efficient adaptation. MECTA [12] improves memory efficiency through an adaptive BN layer. However, reduced computation or memory does not necessarily result in reduced latency. Furthermore, *retraining-free TTA* approaches, including prototype-based methods [16], [17] and consistency regularization [49], enhance efficiency by eliminating backpropagation. However, these methods do not update model parameters, limiting their ability to refine representations. This limitation causes significant accuracy losses in mobile environments with continuous or drastic shifts (discussed in § VII-B1), which motivates our work.

IX. CONCLUSIONS

This paper introduces AdaShadow, a responsive test-time adaptation framework designed for non-stationary mobile environments across CNN and ViT models. It represents the first attempt at near-real-time on-device DL model adaptation without requiring source data or supervision. AdaShadow accurately estimates layer importance during forward pass, calibrates offline latency measurements to runtime dynamics, and efficiently schedules optimal layer updates, addressing latency bottlenecks in mobile contexts without sacrificing accuracy. Evaluations in diverse real-world scenarios demon-

strate significant reductions in adaptation latency and improvements in accuracy. We plan to extend AdaShadow to mobile applications with tighter latency constraints and more complex multimodal tasks. Integrating AdaShadow with federated learning, *e.g.*, federated TTA, could also offer promising avenues for enhancing model robustness and user privacy.

ACKNOWLEDGMENTS

This work was partially supported by the National Science Fund for Distinguished Young Scholars (62025205), the National Natural Science Foundation of China (No. 62032020, 62472354, 62102317), and CityU APRC grant (No. 9610633).

REFERENCES

- [1] S. Liu, B. Guo, C. Fang, Z. Wang, S. Luo, Z. Zhou, and Z. Yu, "Enabling resource-efficient aiot system with cross-level optimization: A survey," *IEEE Communications Surveys & Tutorials*, 2023.
- [2] C. Fang, S. Liu, Z. Zhou, B. Guo, J. Tang, K. Ma, and Z. Yu, "Adashadow: Responsive test-time model adaptation in non-stationary mobile environments," in *Proceedings of the ACM SenSys*, 2024.
- [3] Sweden Data Collection & Processing, 2024.
- [4] Google, <https://developers.google.com/ar>.
- [5] Apple, <https://developer.apple.com/augmented-reality/arkit/>.
- [6] Google, <https://www.google.com/photos/about/>.
- [7] Apple, <https://www.apple.com/apple-intelligence/>.
- [8] L. Liu, S. Lu, R. Zhong, B. Wu, Y. Yao, Q. Zhang, and W. Shi, "Computing systems for autonomous driving: State of the art and challenges," *IEEE Internet of Things Journal*, vol. 8, no. 8, pp. 6469–6486, 2020.
- [9] J. Liang, R. He, and T. Tan, "A comprehensive survey on test-time adaptation under distribution shifts," *arXiv preprint arXiv:2303.15361*, 2023.
- [10] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," in *Proceedings of the ICLR*, 2021.
- [11] Q. Wang, O. Fink, L. Van Gool, and D. Dai, "Continual test-time domain adaptation," in *Proceedings of the IEEE/CVF CVPR*, 2022, pp. 7201–7211.
- [12] J. Hong, L. Lyu, J. Zhou, and M. Spranger, "Mecta: Memory-economic continual test-time model adaptation," in *Proceedings of the ICLR*, 2023.
- [13] S. Niu, J. Wu, Y. Zhang, Y. Chen, S. Zheng, P. Zhao, and M. Tan, "Efficient test-time model adaptation without forgetting," in *Proceedings of the ACM ICML*, 2022, pp. 16888–16905.
- [14] J. Song, J. Lee, I. S. Kweon, and S. Choi, "Ecotta: Memory-efficient continual test-time adaptation via self-distilled regularization," in *Proceedings of the IEEE/CVF CVPR*, 2023, pp. 11920–11929.
- [15] W. Shi, R. Xu, Y. Zhuang, Y. Yu, H. Wu, C. Yang, and M. D. Wang, "Medadapter: Efficient test-time adaptation of large language models towards medical reasoning," *arXiv preprint arXiv:2405.03000*, 2024.
- [16] Y. Wang, A. Cheraghian, Z. Hayder, J. Hong, S. Ramasinghe, S. Rahman, D. Ahmedt-Aristizabal, X. Li, L. Petersson, and M. Harandi, "Backpropagation-free network for 3d test-time adaptation," in *Proceedings of the IEEE/CVF CVPR*, 2024, pp. 23 231–23 241.
- [17] S. Wang, J. Wang, H. Xi, B. Zhang, L. Zhang, and H. Wei, "Optimization-free test-time adaptation for cross-person activity recognition," *Proceedings of the ACM on IMMUT*, vol. 7, no. 4, pp. 1–27, 2024.
- [18] X. Ma, J. Zhang, S. Guo, and W. Xu, "Swapprompt: Test-time prompt adaptation for vision-language models," *Proceedings of the NeurIPS*, vol. 36, 2024.
- [19] K. Huang, B. Yang, and W. Gao, "Elastictrainer: Speeding up on-device training with runtime elastic tensor selection," in *Proceedings of the ACM MobiSys*, 2023, pp. 56–69.
- [20] Y. Lee, A. S. Chen, F. Tajwar, A. Kumar, H. Yao, P. Liang, and C. Finn, "Surgical fine-tuning improves adaptation to distribution shifts," in *Proceedings of the ICLR*, 2023.
- [21] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256kb memory," in *Proceedings of the NeurIPS*, 2022.
- [22] K. Huang, H. Yin, H. Huang, and W. Gao, "Towards green ai in fine-tuning large language models via adaptive backpropagation," *Proceedings of the ICLR*, 2024.
- [23] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Proceedings of the NeurIPS*, 2023.
- [24] Q. Wang, M. Xu, C. Jin, X. Dong, J. Yuan, X. Jin, G. Huang, Y. Liu, and X. Liu, "Melon: Breaking the memory wall for resource-efficient on-device machine learning," in *Proceedings of the ACM MobiSys*, 2022, pp. 450–463.
- [25] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proceedings of the ICLR*, 2021.
- [26] L. L. Zhang, S. Han, J. Wei, N. Zheng, T. Cao, Y. Yang, and Y. Liu, "Nn-meter: Towards accurate latency prediction of deep-learning model inference on diverse edge devices," in *Proceedings of the ACM MobiSys*, 2021, pp. 81–93.
- [27] J. J. Moon, H. S. Lee, J. Chu, D. Park, S. Hong, H. Seo, D. Jeong, S. Kong, and M. Ham, "A new frontier of ai: On-device ai training and personalization," in *Proceedings of the ICSE*, 2024.
- [28] S. Lu, B. Wang, H. Wang, L. Chen, M. Linjian, and X. Zhang, "A real-time object detection algorithm for video," *Computers & Electrical Engineering*, vol. 77, pp. 398–408, 2019.
- [29] S. Petrangeli, G. Simon, H. Wang, and V. Swaminathan, "Dynamic adaptive streaming for augmented reality applications," in *Proceedings of the ISM*. IEEE, 2019, pp. 56–567.
- [30] I. Hasan, S. Liao, J. Li, S. U. Akram, and L. Shao, "Generalizable pedestrian detection: The elephant in the room," in *Proceedings of the IEEE/CVF CVPR*, 2021, pp. 11 328–11 337.
- [31] M. J. Mirza, J. Micorek, H. Possegger, and H. Bischof, "The norm must go on: Dynamic unsupervised domain adaptation by normalization," in *Proceedings of the IEEE/CVF CVPR*, 2022, pp. 14 765–14 775.
- [32] H. She, Y. Luo, Z. Xiang, W. Liang, and Y. Xie, "Accurate latency prediction of deep learning model inference under dynamic runtime resource," in *Proceedings of the ICONIP*. Springer, 2023, pp. 495–510.
- [33] M. Xu, W. Yin, D. Cai, R. Yi, D. Xu, Q. Wang, B. Wu, Y. Zhao, C. Yang, S. Wang, Q. Zhang, Z. Lu, L. Zhang, S. Wang, Y. Li, L. Yunxin, X. Jin, and X. Liu, "A survey of resource-efficient llm and multimodal foundation models," *arXiv preprint arXiv:2401.08092*, 2024.
- [34] H. Chu, X. Zheng, L. Liu, and H. Ma, "nnperf: Demystifying dnn runtime inference latency on mobile platforms," in *Proceedings of the ACM SenSys*, 2023, pp. 125–137.
- [35] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE/CVF CVPR*, 2016, pp. 770–778.
- [36] D. Iorga, T. Sorensen, and A. F. Donaldson, "Do your cores play nicely? a portable framework for multi-core interference tuning and analysis," *arXiv preprint arXiv:1809.05197*, 2018.
- [37] J. Donald and M. Martonosi, "Techniques for multicore thermal management: Classification and new exploration," *ACM SIGARCH computer architecture news*, vol. 34, no. 2, pp. 78–88, 2006.
- [38] U. Verner, A. Schuster, M. Silberstein, and A. Mendelson, "Scheduling processing of real-time data streams on heterogeneous multi-gpu systems," in *Proceedings of the ACM SYSTOR*, 2012, pp. 1–12.
- [39] S. Herbert and D. Marculescu, "Analysis of dynamic voltage/frequency scaling in chip-multiprocessors," in *Proceedings of the ISLPED*, 2007, pp. 38–43.
- [40] C. White, W. Neiswanger, and Y. Savani, "Bananas: Bayesian optimization with neural architectures for neural architecture search," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 12, 2021, pp. 10 293–10 301.
- [41] S. Niu, J. Wu, Y. Zhang, Z. Wen, Y. Chen, P. Zhao, and M. Tan, "Towards stable test-time adaptation in dynamic wild world," 2023.
- [42] X. Zhang, Y. He, R. Xu, H. Yu, Z. Shen, and P. Cui, "Nico++: Towards better benchmarking for domain generalization," in *Proceedings of the IEEE/CVF CVPR*, 2023, pp. 16 036–16 047.
- [43] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," 2019.
- [44] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proceedings of the ICLR*, 2014.
- [45] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE/CVF CVPR*, 2018, pp. 4510–4520.
- [46] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers & distillation through attention," in *Proceedings of the ACM ICML*, 2021, pp. 10 347–10 357.

- [47] S. N. Wadekar and A. Chaurasia, "Mobilevitv3: Mobile-friendly vision transformer with simple and effective fusion of local, global and input features," 2022.
- [48] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 10012–10022.
- [49] M. Boudiaf, R. Mueller, I. Ben Ayed, and L. Bertinetto, "Parameter-free online test-time adaptation," in *Proceedings of the IEEE/CVF CVPR*, 2022, pp. 8344–8353.
- [50] Y. Yu, L. Sheng, R. He, and J. Liang, "Benchmarking test-time adaptation against distribution shifts in image classification," *arXiv preprint arXiv:2307.03133*, 2023.
- [51] W. Liu, Z. Lai, S. Li, Y. Duan, K. Ge, and D. Li, "Autopipe: A fast pipeline parallelism approach with balanced partitioning and micro-batch slicing," in *Proceedings of the IEEE CLUSTER*. IEEE, 2022, pp. 301–312.
- [52] M. Rhu, N. Gimselshein, J. Clemons, A. Zulfiqar, and S. W. Keckler, "vdnn: Virtualized deep neural networks for scalable, memory-efficient neural network design," in *Proceedings of the MICRO*, 2016, pp. 1–13.
- [53] E. B. Zaken, S. Ravfogel, and Y. Goldberg, "Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," *Proceedings of the ACL*, 2022.
- [54] H. Touvron, M. Cord, A. El-Nouby, J. Verbeek, and H. Jégou, "Three things everyone should know about vision transformers," in *Proceedings of the ECCV*, 2022, pp. 497–515.
- [55] S. Huang, B. Gong, Y. Pan, J. Jiang, Y. Lv, Y. Li, and D. Wang, "Vop: Text-video co-operative prompt tuning for cross-modal retrieval," in *Proceedings of the IEEE/CVF CVPR*, 2023, pp. 6565–6574.
- [56] Y. Xin, J. Du, Q. Wang, Z. Lin, and K. Yan, "Vmt-adapter: Parameter-efficient transfer learning for multi-task dense scene understanding," in *Proceedings of the AAAI*, vol. 38, no. 14, 2024, pp. 16 085–16 093.
- [57] M. Chen, H. Xue, and D. Cai, "Domain adaptation for semantic segmentation with maximum squares loss," in *Proceedings of the IEEE/CVF ICCV*, 2019, pp. 2090–2099.
- [58] A. M. N. Taufique, C. S. Jahan, and A. Savakis, "Unsupervised continual learning for gradually varying domains," in *Proceedings of the IEEE/CVF CVPR*, 2022, pp. 3740–3750.



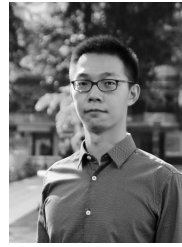
Cheng Fang received the B.E. degree from Northwestern Polytechnical University, Xi'an, China, in 2022, where he is currently pursuing the Master's degree. His research interests include deep learning in resource-constrained devices and mobile model adaptation.



Bin Guo (Senior Member, IEEE) received the Ph.D. degree from Keio University, Japan. He was a Postdoc Researcher with Institut Telecom SudParis, France. He is currently a Professor with Northwestern Polytechnical University, China. His research interests include ubiquitous computing, mobile crowd sensing, and HCI. He served as an Associate Editor for *IEEE Communications Magazine* and *IEEE Transactions on Human-Machine Systems*.



Sicong Liu (Member, IEEE) received the Ph.D. degree from Xidian University in 2020. She is currently an Associate Professor with the School of Computer Science, Northwestern Polytechnical University. Her research interests include mobile computing and resource-efficient mobile deep learning.



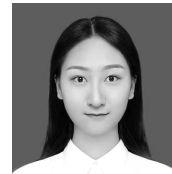
Zimu Zhou (Member, IEEE) received the B.E. degree from Tsinghua University in 2011, and the Ph.D. degree from The Hong Kong University of Science and Technology in 2016. He was a Post-doctoral Researcher at ETH Zurich and an Assistant Professor at Singapore Management University. He is currently an Assistant Professor with the City University of Hong Kong. His research interests include mobile and ubiquitous computing and AIoT.



Jiaqi Tang (Student Member, IEEE) received the B.E. degree from Northwestern Polytechnical University, Xi'an, China, in 2022. He is currently an M.Phil. student at The Hong Kong University of Science and Technology (Guangzhou). His research interests include multimodal large models, industrial vision, and anomaly detection.



Ke Ma received the B.E. degree from Northwestern Polytechnical University in 2021, where he is currently pursuing the Ph.D. degree. His research interests include efficient AI on resource-constrained devices and deep model adaptation.



Shiyan Luo received the B.E. degree from Northwestern Polytechnical University in 2023, where she is currently a Master's student. Her research interests include mobile computing and AIoT systems.



Geyang Song received the B.E. degree from Northwestern Polytechnical University, Xi'an, China, in 2024, where he is currently pursuing the Master's degree. His research interests include the Artificial Intelligence of Things (AIoT).



Zhiwen Yu (Senior Member, IEEE) received the Ph.D. degree from Northwestern Polytechnical University, Xi'an, China, in 2005, where he is currently a Professor and the Dean of the School of Computer Science. He was an Alexander Von Humboldt Fellow with Mannheim University, Germany, and a Research Fellow with Kyoto University, Japan. His research interests include ubiquitous computing, HCI, and mobile sensing.